AI Story Series Studio

Complete production, software and build plan

Prepared for: Ghulam Nabi

Date: 7 October 2026

Version: 1.0 (planning baseline)

> **What this document is.** A single plan for launching an AI-made animated story series on YouTube from Pakistan at zero cash cost, and for the software that automates the production pipeline. It covers the production format, the tool stack and licenses, the compute plan, the software architecture, the configuration system with provider fallbacks and a manual lane, the reference code, the build schedule, the runbook, and the compliance checklist.

Contents

1. Summary and key decisions

1.1 Decisions made in this plan

1.2 What happens first

2. Goals, constraints and non-goals

2.1 Goals

2.2 Constraints

2.3 Non-goals

3. Platform facts as of October 2026

4. Production design

4.1 Series format

4.2 Episode anatomy

4.3 Shot budget and the image-led style

4.4 Quality bar

4.5 The pilot

5. Tool stack and licenses

5.1 Free tiers that must not be used for monetized video

5.2 The Google student plan

6. Compute plan

6.1 Options

6.2 Decision rule

6.3 How the worker runs in each setting

7. Software architecture

7.1 Principles

7.2 Components

7.3 Repository layout

7.4 Data model

7.5 Shot folder layout

7.6 Status lifecycle

7.7 How one shot flows through the system

8. Configuration system

8.1 The files

8.2 providers.yaml

8.3 pipeline.yaml

8.4 .env.example

8.5 Fallback semantics

8.6 How the manual lane behaves

8.7 Storage backends

8.8 Adding a new provider

8.9 Prompts as files

8.10 Recommendations on the open questions

9. Code: configuration, providers and the chain runner

9.1 studio/config.py

9.2 studio/providers/base.py

9.3 studio/providers/registry.py

9.4 studio/chain.py

9.5 studio/providers/manual.py
9.6 LLM adapters

10. Code: generation, workers, assembly and publishing

10.1 ComfyUI client and providers
10.2 Remote queue and the worker
10.3 The breakdown stage
10.4 The generic stage runner
10.5 Quality checks
10.6 Voice adapters
10.7 Assembly
10.8 Vertical cutdowns
10.9 YouTube publisher
10.10 The state store
10.11 The command line
10.12 The review page

11. Stage specifications

11.1 Prompt templates for images and video
11.2 Evidence of human direction

12. Build plan

12.1 Order of work inside each milestone
12.2 Paid upgrades, in order, once revenue arrives

13. Runbook

13.1 One-time setup on the laptop (Windows)
13.2 Accounts
13.3 Producing an episode
13.4 Daily rhythm during production
13.5 Troubleshooting

14. Compliance, licensing and disclosure

14.1 Disclosure per platform
14.2 License table
14.3 What keeps the channel on the right side of the July 2026 YouTube policy

15. Risks and mitigations

16. Appendix

16.1 Command reference
16.2 Dependencies
16.3 Sources consulted for the facts in this plan

1. Summary and key decisions

> **The plan in one paragraph.** We launch an animated story series on YouTube: 30-minute episodes released weekly, a season compilation every six episodes, and three vertical cutdowns per episode for Instagram, TikTok and Shorts. Production costs nothing in cash: every tool is either open source with a commercial license, included in the Google student plan, or free cloud compute. A Python pipeline automates the work from script to published video. Every external service is a provider defined in a config file, every stage has an ordered fallback chain, and the last link in every chain is a manual provider, so a missing API key or an exhausted quota never stops production. Human approval gates after each stage keep quality high and produce the authorship evidence that YouTube's monetization rules now expect.

1.1 Decisions made in this plan

Decision	Choice	Why
Format	30-minute episodes weekly, a season compilation every 6 episodes, 3 vertical cutdowns per episode	Mid-roll ads need 8+ minutes; compilations add watch time; cutdowns funnel viewers to YouTube
Visual style	Image-led: still keyframes with camera moves for about 60% of runtime, generated video for key shots	Cuts video generation by more than half; this is how the top-earning AI story channels work
Cost	Zero cash until revenue: open-source models with commercial licenses, the Google student plan, Kaggle compute	The free tiers of Kling, ElevenLabs, Suno and Runway forbid commercial use or watermark the output
Compute	Kaggle (30 free GPU hours a week) now; one used RTX 3090 24 GB (about Rs 2.2 lac) after the pilot	A 3090 pays for itself against the $500 to $800 per episode that API video would cost
Software	Python pipeline; providers in YAML or JSON; fallback chains; manual lane; human gates; resumable	Swap Gemini for Claude or ChatGPT by editing one line; never blocked; evidence of human direction
Script writing	Paste from any chat app (manual provider) now; Claude or ChatGPT API later	A script is written once per episode, so even a paid API costs cents; the breakdown is where the volume is
Storage	SQLite by default, a single JSON file as the alternative, chosen in config	Both implement the same interface; JSON is easy to read and edit by hand
Revenue platform	YouTube only for now	Facebook ad payouts and TikTok rewards are not available in Pakistan; Instagram pays almost nothing

1.2 What happens first

1. Claim the Google AI student plan (free for a year, redeem before 31 December 2026).
2. Create a free Gemini API key in Google AI Studio. Do not attach billing.
3. Claim the GitHub Student Developer Pack (Copilot Pro and cloud credits for the build).
4. Create the YouTube channel and a Google Cloud project with the YouTube Data API enabled.
5. Write the series bible: three to five characters, locations, style, voice notes.
6. Build milestones M0 to M2 of the software (section 12) and produce the storyboard for a 3 to 5 minute pilot.
7. Build M3 to M4 and finish the pilot end to end on Kaggle.
8. Decide on the GPU purchase with real numbers from the pilot.

2. Goals, constraints and non-goals

2.1 Goals

- A weekly 30-minute episode of an original animated story series, with consistent characters and clean dialogue audio, at 1080p.
- Zero cash spend until the channel earns. Upgrades are bought from revenue, in a planned order.
- No quality compromise that a viewer would notice. The compromise we accept instead is time: more hours of generation and curation.
- A pipeline that one person can run, that resumes after failures, and that keeps every artifact on disk.
- Full control over which API or model does each job, by editing configuration, not code.
- Compliance with the 2026 monetization rules on YouTube, Meta and TikTok for AI-assisted content.

2.2 Constraints

- **Location.** The team is in Pakistan. YouTube's standard Partner Program thresholds apply (1,000 subscribers and 4,000 public watch hours, or 10 million Shorts views). The lowered 500-subscriber tier is not available in Pakistan. Facebook's ad revenue program has required payout and tax details from an eligible country since March 2025, and Pakistan is not one. TikTok's Creator Rewards program is not open in Pakistan.
- **Licensing.** Only tools whose license allows commercial use of the output may touch a monetized video. Section 5 lists what is safe and what is not.
- **Student plan quotas.** The plan's image and video quotas live inside the Gemini app and Flow, which have no API. They are a manual lane, not something the pipeline can call.
- **Compute.** No GPU today. Kaggle gives 30 hours a week on a 16 GB T4, enough for a pilot and for voice work, not for weekly 30-minute episodes.
- **Policy.** YouTube's July 2026 update demonetizes repetitive, mass-produced or clearly inauthentic content and names AI content explicitly. Human-directed AI work remains monetizable. Our gates and evidence folder exist for this reason.

2.3 Non-goals

- Not a content farm. No template videos, no voice cloning of real people, no AI personas giving health, finance or legal advice.
- No automation of Facebook, Instagram or TikTok uploads in the first version. Their APIs require app review, and TikTok's unaudited apps can only post privately. Uploads there stay manual until the channel justifies the review process.
- No browser automation of Flow or the Gemini app to extract the student plan quotas. That violates the terms and risks the account.

3. Platform facts as of October 2026

Platform	Program and thresholds	AI content rules	Pakistan status	Our use
YouTube	Partner Program: 1,000 subscribers and 4,000 watch hours in 12 months, or 10 million Shorts views in 90 days	Since 15 July 2026 "repetitive, mass-produced or clearly inauthentic" content is demonetized; AI as a production tool with human direction stays monetizable; synthetic media disclosure required	Standard thresholds apply; the 500-subscriber tier is not offered; AdSense pays Pakistani banks by wire	Primary revenue platform. Long episodes with mid-rolls, playlists, season compilations
Facebook	Content Monetization: 10,000 followers, 600,000 minutes viewed in 60 days, invite based	AI-assisted original work is fine; fully synthetic mass-produced content is deprioritized	Ad payouts need financial details from an eligible country; Stars (tips) work	Reach only. Native uploads of episodes; no revenue expected
Instagram	Reels bonus ended August 2025; ad revenue share in a few markets with 10,000 followers and 600,000 minutes	AI label required; unlabeled AI content loses reach	No ad program	Teasers and scene cutdowns linking to YouTube
TikTok	Creator Rewards: 10,000 followers, 100,000 views in 30 days, videos of 1 minute or longer	Fully AI-generated videos are excluded from rewards; AI label required	Program not available	Teasers and cutdowns linking to YouTube
Amazon	Prime Video Direct no longer accepts unsolicited shorts or non-fiction	n/a	n/a	Dropped from the plan

Reported earning rates come from creator reports rather than official data, so treat them as rough:

Format	RPM (revenue per 1,000 views)
YouTube Shorts	$0.04 to $0.10
YouTube long-form animated story, mixed audience	$3 to $7
Same, with a mostly US, UK and Canadian audience	$8 to $15

The lesson from those numbers: audience geography and video length matter more than view counts. Write and voice for an English-speaking Tier 1 audience from day one, and make long videos.

4. Production design

4.1 Series format

- **Season:** 6 to 8 episodes with one story arc, a recurring cast of 3 to 5 characters, and 4 to 6 recurring locations. Recurring assets are what make consistency and speed possible.
- **Episode:** about 30 minutes, 12 to 16 scenes, about 300 shots of 5 to 8 seconds. Dialogue-heavy close-ups get lip sync. Everything else is either a generated clip or an animated still.
- **Release:** one episode a week on YouTube. After six episodes, a "full season" compilation with a new cold open and recap. One compilation per season only, so it never reads as re-uploads.
- **Cutdowns:** three 60 to 90 second vertical scenes per episode with burned captions, posted to Instagram, TikTok and Shorts with the AI label and a link to the full episode.

4.2 Episode anatomy

Segment	Length	Purpose
Cold open	60 to 90 s	Hook from inside the story, no logo yet
Title card	5 s	Series name, episode number
Act 1	8 min	Set up the episode's problem
Mid-roll point	at 8 min	First ad slot, so Act 1 ends on a question
Act 2	10 min	Complication, the recurring cast under pressure
Act 3	8 min	Resolution and a cliffhanger line
End card	20 s	Next episode tease, playlist link

4.3 Shot budget and the image-led style

A 30-minute episode is 1,800 seconds. At 6 seconds a shot that is 300 shots. The budget below is what the pipeline aims for.

Shot kind	Share of shots	How it is made	Generation cost
Animated still	about 60%	One approved keyframe with a slow camera move added in the edit	One image
Generated clip	about 30%	Image-to-video from the keyframe, 5 to 8 seconds	3 to 4 attempts per keeper
Dialogue close-up	about 10%	Generated clip plus lip sync from the voice line	3 to 4 attempts plus one lip-sync pass

So an episode needs about 300 keyframes, about 120 generated clips (360 to 480 attempts), and about 30 lip-sync passes. On a 3090 that is roughly 40 to 60 GPU hours of overnight batches. On Kaggle alone it is more than a week of the free quota, which is why the GPU purchase is in the plan.

4.4 Quality bar

- 1920 by 1080 at 24 fps, H.264 at CRF 18, with subtitles burned in and a clean SRT file uploaded alongside.
- Dialogue normalized to -16 LUFS, music ducked under speech, no clipping.
- Characters pass a similarity check against their reference sheet before a keyframe is offered for approval.
- No shot shorter than 4 seconds, no black or frozen frames, no watermarks.
- Every episode has a human-written script, a human-edited breakdown, and human approval of every keyframe and clip.

4.5 The pilot

Before any 30-minute episode, make a 3 to 5 minute pilot through the exact pipeline. It measures the real keeper rate per shot, whether the characters hold across shots, how long Kaggle takes per clip, and how many hours the human work takes. Those four numbers set the budget and the schedule for the season.

5. Tool stack and licenses

Three lanes exist for every capability. The **free lane** is what the pipeline calls by default. The **premium lane** is the Google student plan, used by hand for the shots that matter most. The **paid lane** is what revenue buys later, in the order listed in section 12.

Capability	Free lane (pipeline default)	License	Premium lane (student plan, manual)	Paid lane later
Script writing	Paste from Claude, ChatGPT or Gemini chat apps	n/a	Gemini Pro in the app	Claude or OpenAI API
Breakdown, prompt writing	Gemini API free tier (text only)	free tier terms	Same	Claude or OpenAI API
Character sheets, keyframes	Qwen-Image and Qwen-Image-Edit in ComfyUI; Flux.1 schnell as backup	Apache 2.0	Nano Banana Pro, 50 images a day	Google AI Pro
Character lock	One small LoRA per character trained on its sheet	yours	Reference images in Nano Banana Pro	same
Video	Wan 2.2 (TI2V-5B on 16 GB, 14B on 24 GB); LTX-2.3 for native audio	Apache 2.0; LTX community license	Veo 3.1 Fast, about 10 clips a month	Kling Standard at $10
Voices	Chatterbox (cloning, emotion control); Kokoro (fast, CPU, stock voices)	MIT; Apache 2.0	n/a	ElevenLabs Starter at $6
Lip sync	LatentSync or InfiniteTalk in ComfyUI	Apache 2.0	n/a	same
Music	ACE-Step 1.5 in ComfyUI; YouTube Audio Library	Apache 2.0; free	Lyria 3, 20 tracks a day	Suno Pro at $10
Sound effects	Freesound (CC0), Pixabay	CC0	n/a	same
Upscale	SeedVR2, Real-ESRGAN	open	n/a	Topaz
Edit, assembly	FFmpeg in the pipeline; DaVinci Resolve for manual polish	free	n/a	same
Orchestration of models	ComfyUI as a local server with an HTTP and websocket API	GPL for the server, outputs are yours	n/a	same

5.1 Free tiers that must not be used for monetized video

Tool	Free tier	Problem
Kling	66 credits a day	Watermark; no commercial rights below the $10 plan
ElevenLabs	10,000 credits a month	No commercial license
Suno	50 credits a day	Personal use only; you do not own the songs
Runway	125 credits once	Watermark; never refreshes
Grok Imagine	paywalled since early 2026	Needs X Premium
Flux.1 dev, Flux Kontext dev	open weights	Non-commercial license; use schnell or Qwen instead

5.2 The Google student plan

Pakistani students aged 18 and over get Google AI Plus free for 12 months (some outlets report AI Pro; the offer page shows which). Verification is by student status, and the offer must be redeemed by 31 December 2026. What the Plus plan includes, per Google's May 2026 feature list:

- Nano Banana Pro image generation, 50 images a day, with reference-image consistency. Use it for character sheets, location sheets, and hero keyframes.
- Veo 3.1 Fast video, about 200 credits a month, roughly 10 clips. Use it for the one or two hero shots per episode.
- Lyria 3 music, 20 tracks a day. Use it for the theme and recurring cues.
- Gemini Pro with higher limits, Deep Research for story research, and 200 GB of Drive, which becomes the shared storage for the job queue.

These quotas are in the app and in Flow. The pipeline treats them as the manual lane: a shot marked for the premium lane gets a request file with the prompt and reference images, you generate it in Flow, drop the file into the shot folder, and the pipeline continues.

6. Compute plan

6.1 Options

Option	Cost	Fits	Throughput	Role in the plan
Kaggle	free, 30 GPU hours a week, 16 GB T4, 9-hour sessions	Wan 2.2 5B, Qwen-Image, Chatterbox, LatentSync, ACE-Step	A few hundred low-resolution clips a week at best	Pilot, voices, music, cutdowns
Colab free	free, roughly 15 to 30 hours a week, throttled	Same, less reliable	Lower	Overflow for voices and images
Used RTX 3090 24 GB	about Rs 2.2 lac from a shop with a short warranty; Rs 1.55 to 2.4 lac on OLX	Everything, including Wan 2.2 14B and LTX-2.3	A 30-minute episode in 40 to 60 overnight hours	Main production after the pilot
New RTX 5060 Ti 16 GB	Rs 1.4 to 2.05 lac new with warranty	Wan 2.2 5B natively, 14B only quantized or offloaded	Slower on the big models	Only if a warranty matters more than VRAM
New RTX 4090	Rs 6.4 to 7.1 lac	Everything	Twice a 3090	Not worth the price for this use

6.2 Decision rule

Buy the 3090 when the pilot proves the format and the keeper rate. At that point the GPU replaces about $500 to $800 of API video per episode, so it pays for itself on the first episode. Check the rest of the machine before buying: a 750 W or better power supply, a case with airflow, and a cabinet that can take a 3-slot card. Load shedding is a real risk for 8-hour batches, so the worker saves after every clip and resumes where it stopped.

6.3 How the worker runs in each setting

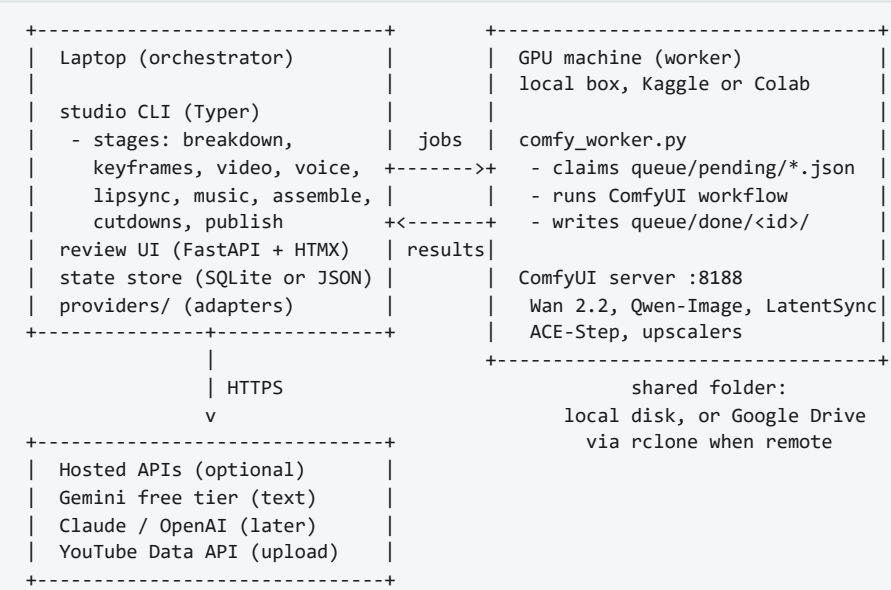
The same worker script runs on Kaggle, on Colab, and on the local GPU. It pulls jobs from a queue folder, runs ComfyUI workflows, and writes results back. On Kaggle the queue folder is synced with Google Drive through rclone at the start and end of each session. On the local GPU the queue is simply a folder on disk. The orchestrator on the laptop never knows or cares where the GPU is.

7. Software architecture

7.1 Principles

1. **Configuration over code.** Every external service is a provider defined in `config/providers.yaml`. Every stage lists its provider chain in `config/pipeline.yaml`. Changing the LLM, the video model or the database is an edit to those files.
2. **Fallback chains.** A stage tries its providers in order. A provider that errors, times out, returns invalid output, or has used its daily quota is skipped. The chain's last entry is always a manual provider.
3. **Manual lane everywhere.** Any job can be completed by a human: paste text, or drop a file with the expected name into the shot folder. The pipeline writes a request file saying exactly what it needs and continues on `studio resume`.
4. **Everything on disk.** Scripts, breakdowns, prompts, candidates, approvals and usage are plain files or a small database inside the project folder. Nothing lives only in memory or only in a web service.
5. **Human gates.** Keyframes, clips and the final cut are approved by a person in a local web page before the next stage uses them. The approval log is the authorship evidence.
6. **Resumable.** Every unit of work has a status. Re-running a stage skips what is approved or done and retries what failed.
7. **Workers anywhere.** GPU work is queued as job files. A worker on Kaggle, Colab or the local GPU claims jobs and returns results through a shared folder.

7.2 Components



7.3 Repository layout

```
ai-series-studio/
  pyproject.toml                package "studio", CLI entry point "studio"
  .env.example                  names of the secrets the config refers to
  config/
    providers.yaml              every provider: type, model, key env var, limits, workflow
    pipeline.yaml               storage backend, worker transport, per-stage chains and QC
  prompts/
    breakdown.md                Jinja templates; editable without touching code
    keyframe.md
    video.md
  workflows/                    ComfyUI workflows exported in API format
    qwen_image_edit.json
```

```
    wan22_i2v_5b.json
    wan22_i2v_14b.json
    latentsync.json
    acestep.json
  bible/
    bible.md                     world, tone, rules, episode formula
    characters/<name>/sheet.png  reference sheets (+ lora.safetensors when trained)
    characters/<name>/voice.wav  10 to 20 s clean reference for Chatterbox
    locations/<name>/sheet.png
    style.md                     palette, lens, lighting words used in every prompt
  episodes/
    ep00-pilot/
      script.md                  the human-written script
      breakdown.json             scenes and shots (validated against the schema)
      shots/s03_07/              one folder per shot (see 7.5)
      audio/                     dialogue track, music, sfx, subtitles
      out/                       ep00-pilot.mp4, cutdowns, evidence.zip
  queue/                         pending/ running/ done/ inputs/  (worker transport)
  state/                         studio.db or studio.json
  studio/                        the Python package (section 9 and 10)
  tests/
  notebooks/kaggle_worker.ipynb  the worker session for Kaggle
```

7.4 Data model

The breakdown is the central document. It is produced by an LLM, validated against a schema, edited by a human, and then drives every later stage.

```
# studio/schemas.py
from typing import Literal
from pydantic import BaseModel, Field

class Line(BaseModel):
    speaker: str                 # a character name from the bible
    text: str
    emotion: str = "neutral"     # passed to the TTS provider

class Shot(BaseModel):
    id: str                      # "s03_07": scene 3, shot 7
    duration_s: float = Field(ge=2, le=12)
    shot_type: Literal["establishing", "wide", "medium", "closeup_dialogue", "insert"]
    characters: list[str] = []
    location: str
    camera: str                  # "slow push-in", "static", "pan left"
    action: str                  # what the viewer sees happen
    lines: list[Line] = []
    needs_video: bool = True     # False: approved keyframe + camera move in the edit
    lane: Literal["free", "premium"] = "free"   # premium = make by hand in Flow

class Scene(BaseModel):
    id: str
    title: str
    location: str
    time_of_day: str
    mood: str                    # drives the music prompt
    shots: list[Shot]

class Breakdown(BaseModel):
    episode: str
    logline: str
    scenes: list[Scene]

BREAKDOWN_SCHEMA = Breakdown.model_json_schema()
```

The state store holds, per shot and per stage, a status and a note, plus a usage ledger per provider per day. Those two tables are all the orchestration needs.

7.5 Shot folder layout

```
episodes/ep00-pilot/shots/s03_07/
  shot.json               the Shot record (copied from the breakdown)
  keyframes/
    cand_1.png ... cand_4.png
    approved.png          symlink or copy of the chosen candidate
    MANUAL_REQUIRED.md    present only while a human step is pending
  video/
    attempt_1.mp4 ... attempt_n.mp4
    approved.mp4
  voice/
    line_1.wav, line_2.wav
  lipsync/
    approved.mp4
  prompts.json            the exact prompts and seeds used, for reproducibility
```

7.6 Status lifecycle

Each shot carries one status per stage:

```
pending -> queued -> generated -> needs_review -> approved -> done
                \-> manual_required (a request file exists) -> resume -> generated
                \-> blocked (every provider in the chain failed; shown in the review UI)
approved + "regenerate" in the UI -> pending again, with the reviewer's note added to the prompt
```

`skipped` marks shots that a stage does not apply to, for example the video stage on an animated still.

7.7 How one shot flows through the system

1. The breakdown stage produces `s03_07` with its action, camera, characters and lines.
2. The keyframe stage renders the keyframe prompt template with the shot and the bible, calls the image chain, gets four candidates, ranks them by similarity to the character sheet, and sets `needs_review`.
3. You approve candidate 2 in the review page. Status becomes `approved`.
4. The video stage builds an image-to-video job from the approved keyframe and the video prompt, writes it to the queue, and sets `queued`.
5. The worker on Kaggle claims it, runs the Wan 2.2 workflow, and writes the clip back. QC checks duration, black frames and freezes. Status becomes `needs_review`.
6. You approve the clip. The voice stage has already produced the two dialogue lines with Chatterbox, so the lip-sync stage runs on the approved clip.
7. Assembly places the clip on the timeline at its scene offset, lays the lines under it, ducks the music, burns the subtitle, and renders the episode.
8. Publish uploads the master with chapters and the synthetic-media flag, as private, for a final watch before it goes public.

8. Configuration system

This section answers the design request directly: APIs, models and the database are all chosen in configuration files, providers can be added or swapped by editing those files, every stage has a fallback chain, and pasting a script or dropping a file is a first-class provider rather than a workaround.

8.1 The files

File	Holds	Who edits it
`config/providers.yaml`	Every provider: its type, model, the name of the env var holding its key, limits, cost, and for ComfyUI providers the workflow file and input mapping	You, whenever you add or change a service
`config/pipeline.yaml`	The storage backend, the worker transport, and for each stage the ordered chain of provider names, attempts, QC thresholds and whether a human gate follows	You, to change which provider a stage uses
`.env`	The secrets themselves, one per line, never committed	You, once per key
`prompts/*.md`	The prompt templates for each LLM stage, with Jinja placeholders	You, to improve output without code changes
`workflows/*.json`	ComfyUI workflows in API format, one per model or task	You, exported from the ComfyUI UI

Both config files may be `.yaml` or `.json`. The loader accepts either, and they parse to the same objects. YAML is recommended because it allows comments. Keys are never placed in config files: a provider names the environment variable that holds its key, and the loader reads the value from `.env`.

8.2 providers.yaml

```yaml
version: 1

providers:
  # ---------- LLM: script writing, breakdown, prompt writing ----------
  gemini_free:
    type: llm.gemini
    enabled: true
    model: gemini-2.5-flash
    api_key_env: GEMINI_API_KEY
    limits: { per_day: 150 }          # stay under the free tier; the chain moves on when reached
    params: { temperature: 0.6 }

  claude:
    type: llm.anthropic
    enabled: false                    # flip to true when you add ANTHROPIC_API_KEY to .env
    model: claude-opus-5-5
    api_key_env: ANTHROPIC_API_KEY
    params: { effort: medium, max_tokens: 16000 }

  openai:
    type: llm.openai                  # also used for any OpenAI-compatible server
    enabled: false
    model: gpt-5
    api_key_env: OPENAI_API_KEY

  ollama_local:
    type: llm.openai                  # Ollama exposes an OpenAI-compatible endpoint
    enabled: false
    model: qwen3:14b
    base_url: http://localhost:11434/v1

  manual_text:
    type: llm.manual                  # paste the answer into response.md / response.json
```

```
  enabled: true

# ---------- Images: character sheets, keyframes ----------
comfy_qwen_image:
  type: image.comfyui
  enabled: true
  base_url: ${COMFY_URL:-http://127.0.0.1:8188}
  workflow: workflows/qwen_image_edit.json
  inputs:                           # logical input -> "node_id.inputs.field" in the workflow
    prompt: "6.inputs.text"
    negative: "7.inputs.text"
    ref_image: "12.inputs.image"
    seed: "3.inputs.seed"
    width: "5.inputs.width"
    height: "5.inputs.height"
  params: { timeout_s: 600 }

manual_image:
  type: image.manual                # Nano Banana Pro in the Gemini app, or any other source
  enabled: true

# ---------- Video ----------
comfy_wan22:
  type: video.comfyui
  enabled: true
  base_url: ${COMFY_URL:-http://127.0.0.1:8188}
  workflow: workflows/wan22_i2v_5b.json
  inputs:
    prompt: "16.inputs.text"
    negative: "17.inputs.text"
    keyframe: "52.inputs.image"
    seed: "3.inputs.seed"
    length: "55.inputs.length"      # frames = duration_s * fps
    width: "55.inputs.width"
    height: "55.inputs.height"
  params: { timeout_s: 2400, fps: 24 }

comfy_ltx23:
  type: video.comfyui
  enabled: false                    # enable once the 3090 is installed
  workflow: workflows/ltx23_i2v.json
  inputs: { prompt: "8.inputs.text", keyframe: "20.inputs.image", seed: "3.inputs.seed" }

kling_api:
  type: video.kling                 # paid lane; adapter added when revenue allows
  enabled: false
  api_key_env: KLING_API_KEY
  cost: { per_second_usd: 0.10 }

manual_video:
  type: video.manual                # Veo 3.1 Fast in Flow, or anything without an API
  enabled: true

# ---------- Voices ----------
chatterbox_local:
  type: tts.chatterbox
  enabled: true
  params: { device: cuda, exaggeration: 0.5, cfg_weight: 0.5 }

kokoro_local:
  type: tts.kokoro
  enabled: true
  params: { device: cpu, voice: af_heart, lang: a }

elevenlabs:
  type: tts.elevenlabs              # paid lane; adapter added later
  enabled: false
  api_key_env: ELEVENLABS_API_KEY

manual_audio:
  type: tts.manual
  enabled: true
```

```
  # ---------- Lip sync, music ----------
  comfy_latentsync:
    type: lipsync.comfyui
    enabled: true
    workflow: workflows/latentsync.json
    inputs: { video: "10.inputs.video", audio: "11.inputs.audio", seed: "3.inputs.seed" }

  comfy_acestep:
    type: music.comfyui
    enabled: true
    workflow: workflows/acestep.json
    inputs: { prompt: "14.inputs.tags", lyrics: "14.inputs.lyrics", seconds: "17.inputs.seconds", seed:
"3.inputs.seed" }

  manual_music:
    type: music.manual                # Lyria 3 on the student plan, or the YouTube Audio Library
    enabled: true

  # ---------- Publishing ----------
  youtube:
    type: publish.youtube
    enabled: true
    params:
      client_secrets: secrets/youtube_client.json
      token: secrets/youtube_token.json
      category_id: "1"                # Film & Animation
```

8.3 pipeline.yaml

```
version: 1

storage:
  backend: sqlite                     # sqlite | json
  path: state/studio.db               # or state/studio.json

workers:
  transport: local                    # local | rclone
  queue_dir: queue
  remote: gdrive:studio/queue         # used only when transport is rclone

stages:
  script:
    chain: [manual_text]              # today: paste the script. Later: [claude, manual_text]

  breakdown:
    chain: [gemini_free, claude, openai, manual_text]   # disabled providers are skipped automatically
    attempts: 2                       # retry the same provider once on invalid JSON before moving on
    params: { target_shot_s: 6 }

  keyframes:
    chain: [comfy_qwen_image, manual_image]
    candidates: 4
    gate: true
    qc: { face_similarity_min: 0.55 }  # against bible/characters/<name>/sheet.png

  video:
    chain: [comfy_wan22, comfy_ltx23, manual_video]
    attempts: 4                       # new seed each attempt; QC decides what counts as a keeper
    gate: true
    qc: { min_duration_s: 4, max_black_ratio: 0.02, freeze_max_s: 1.0 }
    params: { width: 1280, height: 720 }

  voice:
    chain: [chatterbox_local, kokoro_local, manual_audio]
    params: { loudness_lufs: -16 }

  lipsync:
    chain: [comfy_latentsync, manual_video]
    params: { only_shot_types: [closeup_dialogue] }
```

```
music:
  chain: [comfy_acestep, manual_music]
  params: { seconds_per_cue: 90 }

assemble:
  params: { width: 1920, height: 1080, fps: 24, crf: 18, burn_subtitles: true }

cutdowns:
  params: { count: 3, min_s: 60, max_s: 90, aspect: "9:16" }

publish:
  chain: [youtube]
  gate: true
  params: { privacy: private, synthetic_media: true }
```

8.4 .env.example

```
GEMINI_API_KEY=
ANTHROPIC_API_KEY=
OPENAI_API_KEY=
KLING_API_KEY=
ELEVENLABS_API_KEY=
COMFY_URL=http://127.0.0.1:8188
```

8.5 Fallback semantics

The chain runner applies these rules in order for every job:

1. Providers are tried in the order listed. Providers with `enabled: false` are skipped. A provider that is referenced by a stage but not defined in `providers.yaml` fails config validation at startup, not in the middle of a run.
2. Before calling a provider, the runner checks the usage ledger. If the provider has a `per_day` limit and today's successful calls have reached it, the provider is skipped with the note "daily quota reached".
3. A provider raises a typed error with one of these kinds: `auth`, `quota`, `timeout`, `invalid_output`, `transient`, `unavailable`, `refusal`, `qc_failed`. `timeout`, `transient`, `invalid_output` and `qc_failed` are retried on the same provider up to the stage's `attempts`. `auth`, `quota`, `unavailable` and `refusal` move to the next provider immediately.
4. Every call, successful or not, is written to the usage ledger with its units (tokens, seconds generated, characters spoken) and cost, so the review UI can show what each provider did today and what the paid lanes would have cost.
5. A manual provider never fails. It either finds the expected output file and returns it, or it writes a `MANUAL_REQUIRED.md` request next to where the output should go and stops the job with status `manual_required`. Nothing else in the run is affected; other shots continue.
6. If the chain is exhausted without a manual provider at the end, the job is marked `blocked` with every error listed, and the review UI shows it. Putting a manual provider last is therefore the recommended configuration for every stage.
7. `studio run <stage> --provider <name>` overrides the chain for one run, for example to force Claude for one breakdown or to force the manual lane for a hero shot. In the review UI a shot can be moved to the premium lane, which does the same for that shot only.

8.6 How the manual lane behaves

For a text stage, the request file contains the full rendered prompt and says which file to create, `response.md` for prose or `response.json` for structured output. You paste the prompt into any chat app, save the answer into that file, and run `studio resume <episode>`. The runner validates the JSON against the schema exactly as it would validate an API response.

For an image, video or audio stage, the request file lists the prompt, the reference files to use, the parameters such as duration and size, and the expected filename, for example `keyframes/cand_1.png` or `video/attempt_1.mp4`. You generate it in Flow, Nano Banana Pro, or anywhere else, save it under that name, and resume.

The review UI has a page that lists every pending manual request across the episode, with copy buttons for the prompt and a file-drop area that saves the upload under the expected name.

8.7 Storage backends

The store is an interface with two implementations, chosen by `storage.backend`:

- **sqlite** (default): one file, transactional, fast to query, survives crashes. Tables: `shots`, `stage_status`, `usage`, `approvals`.
- **json**: one human-readable file with the same data, written atomically (write to a temp file, then rename). Fine for the pilot and for inspection; slower once an episode has hundreds of shots and thousands of usage rows.

Switching backends is an edit to one line plus a `studio store migrate` command that copies the data across.

8.8 Adding a new provider

1. Write an adapter class with `run(job) -> Result` and `healthcheck() -> bool` in `studio/providers/`. For anything ComfyUI can run, no new class is needed: export the workflow in API format and map its inputs in YAML.
2. Register the type name once in `studio/providers/registry.py`, or call `register("video.kling", "studio.providers.video.kling:KlingVideo")` from a plugin module listed in config.
3. Add the provider block to `providers.yaml` and put its name in a stage chain.
4. Run `studio config check`. It validates the config, pings every enabled provider, and prints the effective chain for each stage.

8.9 Prompts as files

Every LLM stage renders a template from `prompts/` with the bible, the script or shot, and the stage parameters. Changing how shots are described, how long they run, or what style words every image prompt carries is an edit to a Markdown file. The exact rendered prompt is saved into the shot folder for reproducibility.

8.10 Recommendations on the open questions

- **Gemini for the breakdown, a stronger model for the script.** The breakdown is structured extraction with volume, and any current model does it well. The script is one document per episode where quality matters and cost is trivial, so paste it from the best chat model you have access to, and switch the `script` chain to the Claude or OpenAI API when revenue arrives.
- **YAML with JSON accepted.** Comments in the provider file are worth more than the uniformity of JSON. The loader accepts both and the review UI can export either.
- **SQLite by default.** The JSON backend exists so you can open the state in a text editor during the pilot, but SQLite is what you want once there are 300 shots and a usage ledger.
- **Server-side fallback on Claude.** The Claude adapter enables Anthropic's server-side fallback by default, so a request declined by a safety classifier is re-run on another Claude model within the same call. It is a one-line setting in the adapter and can be removed if you prefer the chain to handle it.

9. Code: configuration, providers and the chain runner

The code in sections 9 and 10 is the reference implementation the build follows. It is complete enough to run once the dependencies are installed, and short enough to read in one sitting. Python 3.12. Dependencies are listed in the appendix.

9.1 studio/config.py

```
from __future__ import annotations
import json, os, re
from pathlib import Path
from typing import Any, Literal
import yaml
from pydantic import BaseModel, Field, model_validator

ENV_RE = re.compile(r"\$\{([A-Z0-9_]+)(?::-([^}]*))?\}")   # ${VAR} and ${VAR:-default}

def _expand_env(value: Any) -> Any:
    if isinstance(value, str):
        return ENV_RE.sub(lambda m: os.environ.get(m.group(1), m.group(2) or ""), value)
    if isinstance(value, list):
        return [_expand_env(v) for v in value]
    if isinstance(value, dict):
        return {k: _expand_env(v) for k, v in value.items()}
    return value

class ProviderConfig(BaseModel):
    type: str                                   # "llm.gemini", "video.comfyui", "tts.manual" ...
    enabled: bool = True
    model: str | None = None
    api_key_env: str | None = None              # name of the env var, never the key itself
    base_url: str | None = None
    workflow: str | None = None                 # ComfyUI workflow JSON, API format
    inputs: dict[str, str] = Field(default_factory=dict)    # logical input -> "node.inputs.field"
    limits: dict[str, int] = Field(default_factory=dict)    # {"per_day": 150}
    cost: dict[str, float] = Field(default_factory=dict)    # {"per_second_usd": 0.10}
    params: dict[str, Any] = Field(default_factory=dict)    # free-form, passed to the adapter

    @property
    def capability(self) -> str:
        return self.type.split(".", 1)[0]

    def api_key(self) -> str | None:
        return os.environ.get(self.api_key_env) if self.api_key_env else None

class StageConfig(BaseModel):
    chain: list[str] = Field(default_factory=list)
    attempts: int = 1
    candidates: int = 1
    gate: bool = False
    qc: dict[str, Any] = Field(default_factory=dict)
    params: dict[str, Any] = Field(default_factory=dict)

class StorageConfig(BaseModel):
    backend: Literal["sqlite", "json"] = "sqlite"
    path: str = "state/studio.db"

class WorkersConfig(BaseModel):
    transport: Literal["local", "rclone"] = "local"
    queue_dir: str = "queue"
    remote: str | None = None

class PipelineConfig(BaseModel):
    storage: StorageConfig = StorageConfig()
    workers: WorkersConfig = WorkersConfig()
    stages: dict[str, StageConfig]

class StudioConfig(BaseModel):
```

```
    providers: dict[str, ProviderConfig]
    pipeline: PipelineConfig
    root: Path

    @model_validator(mode="after")
    def _chains_reference_known_providers(self) -> "StudioConfig":
        for stage, cfg in self.pipeline.stages.items():
            for name in cfg.chain:
                if name not in self.providers:
                    raise ValueError(f"stage '{stage}' references unknown provider '{name}'")
                want = {"script": "llm", "breakdown": "llm", "prompts": "llm", "keyframes": "image",
                        "video": "video", "voice": "tts", "lipsync": "lipsync", "music": "music",
                        "publish": "publish"}.get(stage)
                if want and self.providers[name].capability != want:
                    raise ValueError(f"stage '{stage}' needs a {want} provider, '{name}' is
{self.providers[name].type}")
        return self

def _pick(cfg_dir: Path, stem: str) -> Path:
    for ext in (".yaml", ".yml", ".json"):
        if (p := cfg_dir / f"{stem}{ext}").exists():
            return p
    raise FileNotFoundError(f"config/{stem}.yaml or config/{stem}.json not found")

def _load(path: Path) -> dict:
    text = path.read_text(encoding="utf-8")
    data = json.loads(text) if path.suffix == ".json" else yaml.safe_load(text)
    return _expand_env(data or {})

def load_config(root: Path | str = ".") -> StudioConfig:
    root = Path(root).resolve()
    providers = _load(_pick(root / "config", "providers"))["providers"]
    pipeline = _load(_pick(root / "config", "pipeline"))
    return StudioConfig(providers=providers, pipeline=pipeline, root=root)
```

9.2 studio/providers/base.py

One interface for every capability. A provider receives a `Job` and returns a `Result`. Uniformity is what lets the chain runner, the usage ledger and the review UI treat an LLM call and a video render the same way.

```
from __future__ import annotations
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Literal, Protocol, runtime_checkable

ErrorKind = Literal["auth", "quota", "timeout", "invalid_output", "transient",
                    "unavailable", "refusal", "qc_failed"]
RETRY_SAME_PROVIDER = {"timeout", "transient", "invalid_output", "qc_failed"}

class ProviderError(Exception):
    def __init__(self, kind: ErrorKind, message: str, retryable: bool | None = None):
        super().__init__(f"[{kind}] {message}")
        self.kind = kind
        self.retryable = retryable if retryable is not None else kind in RETRY_SAME_PROVIDER

class ManualRequired(Exception):
    """A human must supply the output. The request file explains what and where."""
    def __init__(self, request_file: Path, expected_output: Path):
        super().__init__(f"manual input required, see {request_file}")
        self.request_file, self.expected_output = request_file, expected_output

class Deferred(Exception):
    """The job was queued for a remote worker. Check again on `studio resume`."""
    def __init__(self, job_id: str):
        super().__init__(f"queued as {job_id}")
        self.job_id = job_id

@dataclass
class Job:
    stage: str
```

```
    episode: str
    shot_id: str | None
    workdir: Path                                  # where outputs are written
    prompt: str = ""
    inputs: dict[str, Path] = field(default_factory=dict)   # keyframe, ref_image, audio, voice_ref ...
    params: dict[str, Any] = field(default_factory=dict)    # duration_s, width, seed, expected_filename ...
    schema: dict | None = None                     # JSON schema for structured LLM output

@dataclass
class Result:
    files: list[Path] = field(default_factory=list)
    data: dict | None = None
    provider: str = ""
    units: float = 0.0                             # tokens, seconds generated, characters spoken
    cost_usd: float = 0.0

@runtime_checkable
class Provider(Protocol):
    name: str
    def run(self, job: Job) -> Result: ...
    def healthcheck(self) -> bool: ...
```

9.3 studio/providers/registry.py

```
from importlib import import_module
from studio.config import ProviderConfig, StudioConfig
from studio.providers.base import Provider

_ADAPTERS: dict[str, str] = {
    "llm.gemini":       "studio.providers.llm.gemini:GeminiLLM",
    "llm.anthropic":    "studio.providers.llm.anthropic:ClaudeLLM",
    "llm.openai":       "studio.providers.llm.openai_compat:OpenAILLM",
    "llm.manual":       "studio.providers.manual:ManualText",
    "image.comfyui":    "studio.providers.comfy:ComfyImage",
    "image.manual":     "studio.providers.manual:ManualFile",
    "video.comfyui":    "studio.providers.comfy:ComfyVideo",
    "video.manual":     "studio.providers.manual:ManualFile",
    "lipsync.comfyui":  "studio.providers.comfy:ComfyVideo",
    "music.comfyui":    "studio.providers.comfy:ComfyAudio",
    "music.manual":     "studio.providers.manual:ManualFile",
    "tts.chatterbox":   "studio.providers.tts.chatterbox:ChatterboxTTS",
    "tts.kokoro":       "studio.providers.tts.kokoro:KokoroTTS",
    "tts.manual":       "studio.providers.manual:ManualFile",
    "publish.youtube":  "studio.providers.publish.youtube:YouTubePublisher",
}

def register(type_name: str, dotted_path: str) -> None:
    """Plugins call this to add a provider type without editing the table above."""
    _ADAPTERS[type_name] = dotted_path

def build(name: str, cfg: ProviderConfig) -> Provider:
    try:
        module, cls = _ADAPTERS[cfg.type].split(":")
    except KeyError:
        raise ValueError(f"provider '{name}' has unknown type '{cfg.type}'. Known: {sorted(_ADAPTERS)}")
    return getattr(import_module(module), cls)(name, cfg)

def chain_for(config: StudioConfig, stage: str, force: str | None = None) -> list[Provider]:
    if force:
        return [build(force, config.providers[force])]
    names = config.pipeline.stages[stage].chain
    return [build(n, config.providers[n]) for n in names if config.providers[n].enabled]
```

9.4 studio/chain.py

```
import logging
from studio.providers.base import Deferred, Job, ManualRequired, Provider, ProviderError, Result
from studio.store.base import Store
```

```
log = logging.getLogger("studio.chain")

class StageBlocked(Exception):
    """Every provider in the chain failed and none was manual."""

def run_chain(providers: list[Provider], job: Job, store: Store, attempts: int = 1, qc=None) -> Result:
    errors: list[str] = []
    for p in providers:
        if not store.quota_ok(p.name):
            errors.append(f"{p.name}: daily quota reached")
            continue
        for attempt in range(1, attempts + 1):
            job.params["attempt"] = attempt
            try:
                result = p.run(job)
                if qc is not None:
                    qc(result)                      # raises ProviderError("qc_failed", ...) when it fails
                store.record_usage(p.name, units=result.units, cost_usd=result.cost_usd, ok=True)
                result.provider = p.name
                return result
            except (ManualRequired, Deferred):
                raise                               # a human or a remote worker finishes this job
            except ProviderError as e:
                store.record_usage(p.name, units=0, cost_usd=0, ok=False, kind=e.kind)
                errors.append(f"{p.name} attempt {attempt}: {e}")
                log.warning("%s failed: %s", p.name, e)
                if not e.retryable:
                    break                           # auth, quota, unavailable, refusal: next provider
    raise StageBlocked(f"stage '{job.stage}' exhausted its chain:\n  " + "\n  ".join(errors))
```

9.5 studio/providers/manual.py

```
import json
from pathlib import Path
from studio.providers.base import Job, ManualRequired, Result

REQUEST = "MANUAL_REQUIRED.md"

class _Manual:
    def __init__(self, name, cfg):
        self.name, self.cfg = name, cfg

    def healthcheck(self) -> bool:
        return True

    def expected(self, job: Job) -> str:
        raise NotImplementedError

    def run(self, job: Job) -> Result:
        out = job.workdir / self.expected(job)
        if out.exists() and out.stat().st_size > 0:
            (job.workdir / REQUEST).unlink(missing_ok=True)   # the human delivered; clear the request
            return self.wrap(out, job)
        job.workdir.mkdir(parents=True, exist_ok=True)
        (job.workdir / REQUEST).write_text(self.instructions(job, out), encoding="utf-8")
        raise ManualRequired(job.workdir / REQUEST, out)

    def instructions(self, job: Job, out: Path) -> str:
        refs = "\n".join(f"- {k}: `{v}`" for k, v in job.inputs.items()) or "- none"
        params = "\n".join(f"- {k}: {v}" for k, v in job.params.items() if k != "attempt") or "- none"
        return (f"# Manual step: {job.stage} for {job.shot_id or job.episode}\n\n"
                f"Save your result as: `{out}`\n\n## Prompt\n\n{job.prompt}\n\n"
                f"## Reference files\n{refs}\n\n## Parameters\n{params}\n\n"
                f"Then run: `studio resume {job.episode}`\n")

class ManualText(_Manual):
    def expected(self, job):
        return "response.json" if job.schema else "response.md"
    def wrap(self, out, job):
```

```
        text = out.read_text(encoding="utf-8")
        return Result(files=[out], data=json.loads(text) if job.schema else {"text": text})

class ManualFile(_Manual):
    """Images, video, audio: the stage sets params['expected_filename'], the human saves that file."""
    def expected(self, job):
        return job.params["expected_filename"]
    def wrap(self, out, job):
        return Result(files=[out])
```

9.6 LLM adapters

All three adapters share one helper that turns the model's text into validated JSON when the job carries a schema.

```
# studio/providers/llm/_common.py
import json, re
from studio.providers.base import ProviderError

def parse_json(text: str) -> dict:
    text = text.strip()
    if text.startswith("```"):                        # tolerate fenced output from chat-style models
        text = re.sub(r"^```[a-zA-Z]*\n|\n```$", "", text)
    try:
        return json.loads(text)
    except json.JSONDecodeError as e:
        raise ProviderError("invalid_output", f"model returned invalid JSON: {e}")
```

Gemini (free tier)

```
# studio/providers/llm/gemini.py      pip install google-genai
from google import genai
from google.genai import errors as genai_errors
from studio.providers.base import Job, ProviderError, Result
from studio.providers.llm._common import parse_json

class GeminiLLM:
    def __init__(self, name, cfg):
        self.name, self.cfg = name, cfg
        self.model = cfg.model or "gemini-2.5-flash"
        self._client = None

    def _client_or_raise(self):
        if self._client is None:
            key = self.cfg.api_key()
            if not key:
                raise ProviderError("auth", f"{self.name}: env var {self.cfg.api_key_env} is not set",
retryable=False)
            self._client = genai.Client(api_key=key)
        return self._client

    def healthcheck(self) -> bool:
        try:
            self._client_or_raise().models.get(model=self.model)
            return True
        except Exception:
            return False

    def run(self, job: Job) -> Result:
        config = {"temperature": self.cfg.params.get("temperature", 0.6)}
        if job.schema:
            config["response_mime_type"] = "application/json"
            config["response_json_schema"] = job.schema
        try:
            resp = self._client_or_raise().models.generate_content(
                model=self.model, contents=job.prompt, config=config)
        except genai_errors.APIError as e:
            kind = "quota" if e.code == 429 else "auth" if e.code in (401, 403) else "transient"
            raise ProviderError(kind, f"gemini {e.code}: {e.message}")
        text = resp.text or ""
```

```
        usage = getattr(resp, "usage_metadata", None)
        return Result(data=parse_json(text) if job.schema else {"text": text},
                      units=float(getattr(usage, "total_token_count", 0) or 0))
```

Claude

The adapter uses the official Anthropic SDK. The default model is Claude Opus 5.5, thinking is adaptive by default on that model, and effort is set explicitly because the model's default is `medium`. Structured output uses `output_config.format`, so the breakdown JSON is schema-valid without prompt tricks. Server-side fallback is enabled so a safety-classifier decline is re-run on another Claude model inside the same call; a decline that still comes back is reported as a `refusal` error and the chain moves on.

```
# studio/providers/llm/anthropic.py      pip install anthropic
import anthropic
from studio.providers.base import Job, ProviderError, Result
from studio.providers.llm._common import parse_json

class ClaudeLLM:
    def __init__(self, name, cfg):
        self.name, self.cfg = name, cfg
        self.model = cfg.model or "claude-opus-5-5"
        self._client = None

    def _client_(self):
        if self._client is None:
            key = self.cfg.api_key()        # None: the SDK resolves ANTHROPIC_API_KEY or an `ant auth login` profile
            self._client = anthropic.Anthropic(api_key=key) if key else anthropic.Anthropic()
        return self._client

    def healthcheck(self) -> bool:
        try:
            self._client_().models.retrieve(self.model)
            return True
        except Exception:
            return False

    def run(self, job: Job) -> Result:
        output_config = {"effort": self.cfg.params.get("effort", "medium")}
        if job.schema:
            output_config["format"] = {"type": "json_schema", "schema": job.schema}
        try:
            resp = self._client_().beta.messages.create(
                model=self.model,
                max_tokens=self.cfg.params.get("max_tokens", 16000),
                output_config=output_config,
                betas=["server-side-fallback-2026-07-01"],
                fallbacks="default",                # re-run on another Claude model if a classifier declines
                messages=[{"role": "user", "content": job.prompt}],
            )
        except anthropic.RateLimitError as e:
            raise ProviderError("quota", f"claude rate limited: {e.message}")
        except anthropic.AuthenticationError as e:
            raise ProviderError("auth", f"claude auth: {e.message}", retryable=False)
        except anthropic.APIStatusError as e:
            raise ProviderError("transient" if e.status_code >= 500 else "unavailable",
                                f"claude {e.status_code}: {e.message}", retryable=e.status_code >= 500)
        except anthropic.APIConnectionError as e:
            raise ProviderError("transient", f"claude connection: {e}")
        if resp.stop_reason == "refusal":
            raise ProviderError("refusal", "claude declined this request", retryable=False)
        text = next((b.text for b in resp.content if b.type == "text"), "")
        return Result(data=parse_json(text) if job.schema else {"text": text},
                      units=float(resp.usage.input_tokens + resp.usage.output_tokens))
```

OpenAI and OpenAI-compatible servers (Ollama, LM Studio)

```
# studio/providers/llm/openai_compat.py      pip install openai
from openai import OpenAI, APIConnectionError, APIStatusError, AuthenticationError, RateLimitError
```

```
from studio.providers.base import Job, ProviderError, Result
from studio.providers.llm._common import parse_json

class OpenAILLM:
    def __init__(self, name, cfg):
        self.name, self.cfg = name, cfg
        self.model = cfg.model or "gpt-5"
        self._client = None

    def _client_(self):
        if self._client is None:
            # base_url None -> api.openai.com; a local server needs no real key
            self._client = OpenAI(api_key=self.cfg.api_key() or "local", base_url=self.cfg.base_url)
        return self._client

    def healthcheck(self) -> bool:
        try:
            self._client_().models.list()
            return True
        except Exception:
            return False

    def run(self, job: Job) -> Result:
        kwargs = {"model": self.model, "messages": [{"role": "user", "content": job.prompt}]}
        if job.schema:
            kwargs["response_format"] = {"type": "json_schema",
                                         "json_schema": {"name": "output", "schema": job.schema}}
        try:
            resp = self._client_().chat.completions.create(**kwargs)
        except RateLimitError as e:
            raise ProviderError("quota", str(e))
        except AuthenticationError as e:
            raise ProviderError("auth", str(e), retryable=False)
        except APIStatusError as e:
            raise ProviderError("transient" if e.status_code >= 500 else "unavailable", str(e),
                                retryable=e.status_code >= 500)
        except APIConnectionError as e:
            raise ProviderError("transient", str(e))
        text = resp.choices[0].message.content or ""
        units = float(resp.usage.total_tokens) if resp.usage else 0.0
        return Result(data=parse_json(text) if job.schema else {"text": text}, units=units)
```

Local servers vary in how well they honor `response_format`. The chain validates the JSON against the schema regardless, and an invalid answer is retried, then passed to the next provider.

10. Code: generation, workers, assembly and publishing

10.1 ComfyUI client and providers

Every local model runs inside ComfyUI, which exposes an HTTP API for queueing a workflow and a websocket for progress. A provider is a workflow file plus a mapping from logical inputs to node fields, so adding a model never needs new Python.

```
# studio/providers/comfy.py      pip install websocket-client
import json, os, time, urllib.error, urllib.parse, urllib.request, uuid
from pathlib import Path
import websocket
from studio.providers.base import Job, ProviderError, Result

def set_input(workflow: dict, path: str, value) -> None:
    """'12.inputs.image' -> workflow['12']['inputs']['image'] = value"""
    node, _, field = path.partition(".inputs.")
    workflow[node]["inputs"][field] = value

class ComfyClient:
    def __init__(self, base_url: str, timeout_s: int = 1800):
        self.base = base_url.rstrip("/")
        self.ws_url = self.base.replace("http", "ws", 1) + "/ws"
        self.timeout_s = timeout_s

    def upload(self, path: Path) -> str:
        boundary = uuid.uuid4().hex
        head = (f"--{boundary}\r\nContent-Disposition: form-data; name=\"image\"; "
                f"filename=\"{path.name}\"\r\nContent-Type: application/octet-stream\r\n\r\n").encode()
        body = head + path.read_bytes() + f"\r\n--{boundary}--\r\n".encode()
        req = urllib.request.Request(f"{self.base}/upload/image", data=body,
                                     headers={"Content-Type": f"multipart/form-data; boundary={boundary}"})
        with urllib.request.urlopen(req, timeout=300) as r:
            return json.load(r)["name"]

    def run(self, workflow: dict, out_dir: Path) -> list[Path]:
        client_id = uuid.uuid4().hex
        ws = websocket.create_connection(f"{self.ws_url}?clientId={client_id}", timeout=self.timeout_s)
        payload = json.dumps({"prompt": workflow, "client_id": client_id}).encode()
        req = urllib.request.Request(f"{self.base}/prompt", data=payload, headers={"Content-Type":
"application/json"})
        try:
            with urllib.request.urlopen(req, timeout=60) as r:
                prompt_id = json.load(r)["prompt_id"]
        except urllib.error.HTTPError as e:
            raise ProviderError("invalid_output", f"ComfyUI rejected the workflow: {e.read()[:400]!r}",
retryable=False)
        deadline = time.time() + self.timeout_s
        try:
            while time.time() < deadline:
                msg = ws.recv()
                if isinstance(msg, bytes):
                    continue                                  # preview frames
                evt = json.loads(msg)
                data = evt.get("data", {})
                if data.get("prompt_id") != prompt_id:
                    continue
                if evt["type"] == "executing" and data.get("node") is None:
                    break                                     # finished
                if evt["type"] == "execution_error":
                    raise ProviderError("transient", f"ComfyUI: {data.get('exception_message')}")
            else:
                raise ProviderError("timeout", "ComfyUI generation timed out")
        finally:
            ws.close()
        with urllib.request.urlopen(f"{self.base}/history/{prompt_id}", timeout=60) as r:
            outputs = json.load(r)[prompt_id]["outputs"]
        files: list[Path] = []
```

```
        out_dir.mkdir(parents=True, exist_ok=True)
        for node in outputs.values():
            for kind in ("images", "gifs", "videos", "audio"):
                for item in node.get(kind, []):
                    q = urllib.parse.urlencode({"filename": item["filename"],
                                                "subfolder": item.get("subfolder", ""),
                                                "type": item.get("type", "output")})
                    dest = out_dir / item["filename"]
                    with urllib.request.urlopen(f"{self.base}/view?{q}", timeout=600) as r:
                        dest.write_bytes(r.read())
                    files.append(dest)
        if not files:
            raise ProviderError("invalid_output", "ComfyUI produced no output files")
        return files

class ComfyBase:
    def __init__(self, name, cfg):
        self.name, self.cfg = name, cfg
        self.client = ComfyClient(cfg.base_url or "http://127.0.0.1:8188", cfg.params.get("timeout_s", 1800))
        self.template = json.loads(Path(cfg.workflow).read_text(encoding="utf-8"))

    def healthcheck(self) -> bool:
        try:
            urllib.request.urlopen(f"{self.client.base}/system_stats", timeout=5)
            return True
        except Exception:
            return False

    def build_workflow(self, job: Job) -> dict:
        wf = json.loads(json.dumps(self.template))           # deep copy
        m = self.cfg.inputs
        if "prompt" in m:   set_input(wf, m["prompt"], job.prompt)
        if "negative" in m: set_input(wf, m["negative"], job.params.get("negative", ""))
        if "seed" in m:     set_input(wf, m["seed"], job.params.get("seed", int.from_bytes(os.urandom(4), "big")))
        for logical, src in job.inputs.items():              # keyframe, ref_image, video, audio ...
            if logical in m:
                set_input(wf, m[logical], self.client.upload(src))
        for key in ("width", "height", "length", "fps", "steps", "seconds", "lyrics"):
            if key in m and key in job.params:
                set_input(wf, m[key], job.params[key])
        return wf

    def run(self, job: Job) -> Result:
        files = self.client.run(self.build_workflow(job), job.workdir)
        if "expected_filename" in job.params:                 # keep the same names the manual lane uses
            dest = job.workdir / job.params["expected_filename"]
            files[0].replace(dest)
            files[0] = dest
        return Result(files=files, units=self.units(job))

class ComfyImage(ComfyBase):
    def units(self, job): return 1.0
class ComfyVideo(ComfyBase):
    def units(self, job): return float(job.params.get("duration_s", 5))
class ComfyAudio(ComfyBase):
    def units(self, job): return float(job.params.get("seconds", 60))
```

10.2 Remote queue and the worker

When `workers.transport` is `rclone`, the video, lip-sync and music providers do not call ComfyUI directly. A wrapper writes the job to the queue folder and defers; the worker runs it on whatever GPU it has; `studio resume` collects the result.

```
# studio/providers/remote.py
import json, shutil, uuid
from pathlib import Path
from studio.providers.base import Deferred, Job, ProviderError, Result

class RemoteQueue:
```

```
    """Wraps a ComfyBase provider: the workflow is built here, executed on the worker."""
    def __init__(self, inner, queue_dir: Path):
        self.inner, self.queue, self.name = inner, queue_dir, inner.name

    def healthcheck(self): return True

    def run(self, job: Job) -> Result:
        marker = job.workdir / ".queued"
        if marker.exists():
            done = self.queue / "done" / marker.read_text()
            result = done / "result.json"
            if not result.exists():
                raise Deferred(marker.read_text())            # still running or not synced yet
            info = json.loads(result.read_text())
            marker.unlink()
            if not info["ok"]:
                raise ProviderError("transient", f"worker: {info['error']}")
            files = []
            for name in info["files"]:
                dest = job.workdir / (job.params.get("expected_filename") or name)
                shutil.move(done / name, dest)
                files.append(dest)
            return Result(files=files, units=self.inner.units(job))
        job_id = uuid.uuid4().hex[:12]
        wf = self.inner.template | {}                         # inputs are resolved on the worker
        inputs_dir = self.queue / "inputs"; inputs_dir.mkdir(parents=True, exist_ok=True)
        spec = {"id": job_id, "workflow": Path(self.inner.cfg.workflow).name, "set": {}, "files": {}}
        m = self.inner.cfg.inputs
        spec["set"][m["prompt"]] = job.prompt
        for logical, src in job.inputs.items():
            if logical in m:
                dest = inputs_dir / f"{job_id}_{src.name}"; shutil.copy(src, dest)
                spec["files"][m[logical]] = dest.name
        for key in ("seed", "width", "height", "length", "seconds", "negative"):
            if key in m and key in job.params:
                spec["set"][m[key]] = job.params[key]
        (self.queue / "pending").mkdir(parents=True, exist_ok=True)
        (self.queue / "pending" / f"{job_id}.json").write_text(json.dumps(spec, indent=2))
        job.workdir.mkdir(parents=True, exist_ok=True)
        marker.write_text(job_id)
        raise Deferred(job_id)
```

```
# studio/workers/comfy_worker.py     runs on the GPU machine or inside a Kaggle notebook
import argparse, json, time
from pathlib import Path
from studio.providers.comfy import ComfyClient, set_input

def serve(queue: Path, comfy_url: str, workflows: Path, once: bool, poll_s: int = 15):
    client = ComfyClient(comfy_url)
    for d in ("pending", "running", "done", "inputs"):
        (queue / d).mkdir(parents=True, exist_ok=True)
    while True:
        jobs = sorted((queue / "pending").glob("*.json"))
        for job_file in jobs:
            running = queue / "running" / job_file.name
            try:
                job_file.rename(running)                      # atomic claim; a second worker loses the race
            except FileNotFoundError:
                continue
            spec = json.loads(running.read_text())
            out = queue / "done" / spec["id"]; out.mkdir(parents=True, exist_ok=True)
            try:
                wf = json.loads((workflows / spec["workflow"]).read_text())
                for path, value in spec["set"].items():
                    set_input(wf, path, value)
                for path, fname in spec["files"].items():
                    set_input(wf, path, client.upload(queue / "inputs" / fname))
                files = client.run(wf, out)
                (out / "result.json").write_text(json.dumps({"ok": True, "files": [f.name for f in files]}))
            except Exception as e:
                (out / "result.json").write_text(json.dumps({"ok": False, "error": str(e)}))
```

```
            finally:
                running.unlink(missing_ok=True)
        if once and not jobs:
            return
        time.sleep(poll_s)

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--queue", required=True); ap.add_argument("--comfy", default="http://127.0.0.1:8188")
    ap.add_argument("--workflows", default="workflows"); ap.add_argument("--once", action="store_true")
    a = ap.parse_args()
    serve(Path(a.queue), a.comfy, Path(a.workflows), a.once)
```

The Kaggle notebook is five cells: clone the repo and ComfyUI, download the model files with `scripts/get_models.sh`, start ComfyUI in the background, `rclone sync` the queue down from Drive, run the worker with `--once`, and `rclone sync` the `done` folder back. The rclone config lives in a Kaggle secret. On the laptop, `studio sync` runs the matching two rclone commands before and after `studio resume`.

10.3 The breakdown stage

```
# studio/stages/breakdown.py      pip install jinja2
from jinja2 import Template
from studio.chain import run_chain
from studio.providers.base import Job
from studio.providers.registry import chain_for
from studio.schemas import BREAKDOWN_SCHEMA, Breakdown

def run(config, store, episode: str, force: str | None = None) -> Breakdown:
    ep = config.root / "episodes" / episode
    script = (ep / "script.md").read_text(encoding="utf-8")
    bible = (config.root / "bible" / "bible.md").read_text(encoding="utf-8")
    style = (config.root / "bible" / "style.md").read_text(encoding="utf-8")
    stage = config.pipeline.stages["breakdown"]
    prompt = Template((config.root / "prompts" / "breakdown.md").read_text(encoding="utf-8")).render(
        episode=episode, script=script, bible=bible, style=style, **stage.params)
    job = Job(stage="breakdown", episode=episode, shot_id=None, workdir=ep / "breakdown",
              prompt=prompt, schema=BREAKDOWN_SCHEMA)
    (job.workdir).mkdir(parents=True, exist_ok=True)
    (job.workdir / "prompt.md").write_text(prompt, encoding="utf-8")     # evidence and reproducibility
    result = run_chain(chain_for(config, "breakdown", force), job, store, attempts=stage.attempts)
    breakdown = Breakdown.model_validate(result.data)                    # fails loudly if the model drifted
    (ep / "breakdown.json").write_text(breakdown.model_dump_json(indent=2), encoding="utf-8")
    for scene in breakdown.scenes:
        for shot in scene.shots:
            store.upsert_shot(episode, shot, scene.id)
    return breakdown
```

```
# prompts/breakdown.md
You are the storyboard supervisor of an animated series. Break the script below into scenes
and shots for production.

SERIES BIBLE
{{ bible }}

VISUAL STYLE
{{ style }}

SCRIPT FOR {{ episode }}
{{ script }}

Rules:
- Shots run 4 to 8 seconds, target {{ target_shot_s }}. Every line of dialogue belongs to exactly one shot.
- Use shot_type closeup_dialogue for any line longer than six words, so it can be lip-synced.
- Set needs_video to false for reaction shots, establishing shots and inserts that a slow camera move
  can carry. Aim for about 60 percent of shots with needs_video false.
- Use only character and location names that appear in the bible, spelled exactly as there.
- camera is a short phrase a cinematographer would say. action describes what the viewer sees, not feelings.
```

```
- Give every scene a mood word usable as a music prompt.
Return JSON that matches the provided schema and nothing else.
```

10.4 The generic stage runner

Keyframes, video, voice, lip sync and music all run through one function. It builds a job per shot, runs the chain, applies QC, and records status. Manual and deferred jobs are collected and reported at the end rather than stopping the run.

```
# studio/stages/runner.py
from studio.chain import StageBlocked, run_chain
from studio.providers.base import Deferred, ManualRequired
from studio.providers.registry import chain_for
from studio.providers.remote import RemoteQueue
from studio.stages.jobs import build_job            # per-stage prompt rendering and input selection
from studio.qc import make_qc

REMOTE_STAGES = {"video", "lipsync", "music"}

def run_stage(config, store, stage: str, episode: str, shot_ids=None, force=None) -> dict:
    scfg = config.pipeline.stages[stage]
    providers = chain_for(config, stage, force)
    if config.pipeline.workers.transport == "rclone" and stage in REMOTE_STAGES:
        queue = config.root / config.pipeline.workers.queue_dir
        providers = [RemoteQueue(p, queue) if hasattr(p, "template") else p for p in providers]
    qc = make_qc(stage, scfg.qc, config)
    report = {"done": [], "review": [], "manual": [], "queued": [], "blocked": [], "skipped": []}
    for shot in store.list_shots(episode):
        sid = shot["id"]
        if shot_ids and sid not in shot_ids:
            continue
        status = shot["stages"].get(stage, {}).get("status")
        if status in ("approved", "done"):
            continue
        if stage == "video" and not shot["needs_video"]:
            store.set_status(episode, sid, stage, "skipped", "animated still"); report["skipped"].append(sid);
continue
        if stage == "lipsync" and shot["shot_type"] not in scfg.params.get("only_shot_types", []):
            store.set_status(episode, sid, stage, "skipped"); report["skipped"].append(sid); continue
        chain = providers
        if shot.get("lane") == "premium":                    # hero shot: only the manual lane
            chain = [p for p in providers if p.name.startswith("manual")] or providers[-1:]
        for n in range(1, (scfg.candidates if stage == "keyframes" else 1) + 1):
            job = build_job(config, stage, episode, shot, candidate=n)
            try:
                result = run_chain(chain, job, store, attempts=scfg.attempts, qc=qc)
                store.set_status(episode, sid, stage, "needs_review" if scfg.gate else "done", result.provider)
                report["review" if scfg.gate else "done"].append(sid)
            except ManualRequired as e:
                store.set_status(episode, sid, stage, "manual_required", str(e.request_file));
report["manual"].append(sid); break
            except Deferred as e:
                store.set_status(episode, sid, stage, "queued", e.job_id); report["queued"].append(sid); break
            except StageBlocked as e:
                store.set_status(episode, sid, stage, "blocked", str(e)); report["blocked"].append(sid); break
    return report

def resume(config, store, episode: str) -> dict:
    """Re-run every stage; manual and queued jobs pick up their results if they now exist."""
    merged = {}
    for stage in ("breakdown", "keyframes", "video", "voice", "lipsync", "music"):
        if stage == "breakdown":
            continue
        for k, v in run_stage(config, store, stage, episode).items():
            merged.setdefault(k, []).extend(v)
    return merged
```

10.5 Quality checks

```
# studio/qc.py
import json, re, subprocess
from pathlib import Path
from studio.providers.base import ProviderError, Result

def ffprobe(path: Path) -> dict:
    out = subprocess.run(["ffprobe", "-v", "quiet", "-print_format", "json", "-show_format", "-show_streams",
                          str(path)], capture_output=True, text=True, check=True).stdout
    return json.loads(out)

def video_qc(min_duration_s=4.0, max_black_ratio=0.02, freeze_max_s=1.0):
    def check(result: Result):
        clip = next((f for f in result.files if f.suffix.lower() in (".mp4", ".webm", ".mov")), None)
        if clip is None:
            raise ProviderError("qc_failed", "no video file in result")
        dur = float(ffprobe(clip)["format"]["duration"])
        if dur < min_duration_s:
            raise ProviderError("qc_failed", f"clip is {dur:.1f}s, under {min_duration_s}s")
        log = subprocess.run(["ffmpeg", "-i", str(clip), "-vf",
                              f"blackdetect=d=0.1:pic_th=0.98,freezedetect=n=-60dB:d={freeze_max_s}",
                              "-an", "-f", "null", "-"], capture_output=True, text=True).stderr
        black = sum(float(x) for x in re.findall(r"black_duration:([\d.]+)", log))
        if black / dur > max_black_ratio:
            raise ProviderError("qc_failed", f"{black:.1f}s of black frames in {dur:.1f}s")
        if "freeze_start" in log:
            raise ProviderError("qc_failed", "frozen frames detected")
    return check

def face_similarity(candidate: Path, reference: Path) -> float:
    """Cosine similarity of face embeddings, 1.0 is identical. pip install insightface onnxruntime opencv-python.
    Works for realistic faces. For stylized characters use CLIP image similarity instead (open_clip)."""
    import cv2, numpy as np
    from insightface.app import FaceAnalysis
    app = FaceAnalysis(providers=["CPUExecutionProvider"]); app.prepare(ctx_id=0, det_size=(640, 640))
    def embed(p):
        faces = app.get(cv2.imread(str(p)))
        return max(faces, key=lambda f: f.det_score).normed_embedding if faces else None
    a, b = embed(candidate), embed(reference)
    return float(np.dot(a, b)) if a is not None and b is not None else 0.0

def make_qc(stage: str, cfg: dict, config):
    if stage == "video":
        return video_qc(cfg.get("min_duration_s", 4), cfg.get("max_black_ratio", 0.02), cfg.get("freeze_max_s", 1.0))
    if stage == "keyframes" and cfg.get("face_similarity_min"):
        def check(result: Result):
            ref = result.data and result.data.get("reference")      # set by build_job from the shot's first
character
            if ref and face_similarity(result.files[0], Path(ref)) < cfg["face_similarity_min"]:
                raise ProviderError("qc_failed", "character does not match the reference sheet")
        return check
    return None
```

10.6 Voice adapters

```
# studio/providers/tts/chatterbox.py      pip install chatterbox-tts torchaudio
import torchaudio as ta
from chatterbox.tts import ChatterboxTTS as _CB
from studio.providers.base import Job, Result

class ChatterboxTTS:
    def __init__(self, name, cfg):
        self.name, self.cfg, self._model = name, cfg, None
    def _m(self):
        if self._model is None:
            self._model = _CB.from_pretrained(device=self.cfg.params.get("device", "cuda"))
        return self._model
    def healthcheck(self): return True
```

```
    def run(self, job: Job) -> Result:
        ref = job.inputs.get("voice_ref")                    # bible/characters/<name>/voice.wav
        wav = self._m().generate(job.prompt, audio_prompt_path=str(ref) if ref else None,
                                 exaggeration=job.params.get("exaggeration", self.cfg.params.get("exaggeration",
0.5)),
                                 cfg_weight=self.cfg.params.get("cfg_weight", 0.5))
        job.workdir.mkdir(parents=True, exist_ok=True)
        out = job.workdir / job.params["expected_filename"]
        ta.save(str(out), wav, self._m().sr)
        return Result(files=[out], units=float(len(job.prompt)))
```

```
# studio/providers/tts/kokoro.py      pip install kokoro soundfile   (runs on CPU, no cloning)
import numpy as np, soundfile as sf
from kokoro import KPipeline
from studio.providers.base import Job, Result

class KokoroTTS:
    def __init__(self, name, cfg):
        self.name, self.cfg = name, cfg
        self.pipe = KPipeline(lang_code=cfg.params.get("lang", "a"))
    def healthcheck(self): return True
    def run(self, job: Job) -> Result:
        voice = job.params.get("voice") or self.cfg.params.get("voice", "af_heart")
        audio = np.concatenate([a for _, _, a in self.pipe(job.prompt, voice=voice)])
        job.workdir.mkdir(parents=True, exist_ok=True)
        out = job.workdir / job.params["expected_filename"]
        sf.write(str(out), audio, 24000)
        return Result(files=[out], units=float(len(job.prompt)))
```

Emotion is carried from the breakdown's `emotion` field into the Chatterbox `exaggeration` parameter through a small table in `build_job` (calm 0.3, neutral 0.5, excited 0.7, angry 0.8).

10.7 Assembly

Assembly has four steps: normalize every approved shot into a uniform clip, build the dialogue track and subtitles from the shot timeline, concatenate, and mix. All of it is FFmpeg; nothing is re-encoded twice.

```
# studio/stages/assemble.py      pip install pydub
import subprocess
from pathlib import Path
from pydub import AudioSegment

def render_shot(shot: dict, src: Path, out: Path, w=1920, h=1080, fps=24):
    """Stills get a slow push-in (Ken Burns). Clips are scaled and padded to the master size."""
    if src.suffix.lower() in (".png", ".jpg", ".jpeg", ".webp"):
        frames = int(shot["duration_s"] * fps)
        vf = (f"scale=8000:-1,zoompan=z='min(zoom+0.0008,1.12)':d={frames}"
              f":x='iw/2-(iw/zoom/2)':y='ih/2-(ih/zoom/2)':s={w}x{h}:fps={fps}")
        cmd = ["ffmpeg", "-y", "-loop", "1", "-i", str(src), "-vf", vf, "-t", str(shot["duration_s"])]
    else:
        vf = f"scale={w}:{h}:force_original_aspect_ratio=decrease,pad={w}:{h}:(ow-iw)/2:(oh-ih)/2,fps={fps}"
        cmd = ["ffmpeg", "-y", "-i", str(src), "-vf", vf, "-t", str(shot["duration_s"])]
    subprocess.run(cmd + ["-an", "-c:v", "libx264", "-preset", "medium", "-crf", "18",
                          "-pix_fmt", "yuv420p", str(out)], check=True)

def timeline(shots: list[dict]) -> list[tuple[dict, float]]:
    """Shots in scene order with their start offset in seconds."""
    t, rows = 0.0, []
    for s in shots:
        rows.append((s, t)); t += s["duration_s"]
    return rows

def dialogue_track(rows, voices_dir: Path, out: Path, gap_ms=250):
    total_ms = int(sum(s["duration_s"] for s, _ in rows) * 1000) + 1000
    track = AudioSegment.silent(duration=total_ms)
    srt, n = [], 1
    for shot, start in rows:
        cursor = int(start * 1000) + 300                      # lines start 0.3 s into the shot
```

```
        for i, line in enumerate(shot.get("lines", []), 1):
            seg = AudioSegment.from_file(voices_dir / shot["id"] / f"line_{i}.wav")
            track = track.overlay(seg, position=cursor)
            srt.append(f"{n}\n{_ts(cursor)} --> {_ts(cursor + len(seg))}\n{line['speaker']}: {line['text']}\n"); n +=
1
            cursor += len(seg) + gap_ms
    track.export(out, format="wav")
    out.with_suffix(".srt").write_text("\n".join(srt), encoding="utf-8")

def _ts(ms: int) -> str:
    h, ms = divmod(ms, 3600000); m, ms = divmod(ms, 60000); s, ms = divmod(ms, 1000)
    return f"{h:02}:{m:02}:{s:02},{ms:03}"

def concat(clips: list[Path], out: Path):
    lst = out.with_suffix(".txt")
    lst.write_text("".join(f"file '{p.as_posix()}'\n" for p in clips), encoding="utf-8")
    subprocess.run(["ffmpeg", "-y", "-f", "concat", "-safe", "0", "-i", str(lst), "-c", "copy", str(out)],
check=True)

def mix_and_master(video: Path, dialogue: Path, music: Path, sfx: Path, subs: Path, out: Path, crf=18):
    """Music ducks under dialogue, SFX mixed in, loudness to -16 LUFS, subtitles burned, faststart for web."""
    fc = ("[1:a]asplit=2[d1][d2];"
          "[2:a]volume=0.9[m];[m][d1]sidechaincompress=threshold=0.05:ratio=8:attack=20:release=400[ducked];"
          "[d2][ducked][3:a]amix=inputs=3:duration=first:normalize=0,loudnorm=I=-16:TP=-1.5:LRA=11[a];"
          f"[0:v]subtitles={subs.as_posix()}:force_style='FontName=Arial,FontSize=20,Outline=1'[v]")
    subprocess.run(["ffmpeg", "-y", "-i", str(video), "-i", str(dialogue), "-i", str(music), "-i", str(sfx),
                    "-filter_complex", fc, "-map", "[v]", "-map", "[a]",
                    "-c:v", "libx264", "-crf", str(crf), "-c:a", "aac", "-b:a", "192k",
                    "-movflags", "+faststart", str(out)], check=True)
```

The music track is built the same way as the dialogue track: one cue per scene, placed at the scene's start, cross-faded by pydub. Title and end cards are two extra "shots" with PNG sources, so they pass through `render_shot` like everything else.

10.8 Vertical cutdowns

```
# studio/stages/cutdowns.py
def pick_scenes(breakdown, count=3, min_s=60, max_s=90):
    """Scenes ranked by dialogue density, trimmed to the length window."""
    scored = []
    for scene in breakdown.scenes:
        dur = sum(s.duration_s for s in scene.shots)
        lines = sum(len(s.lines) for s in scene.shots)
        if dur >= min_s:
            scored.append((lines / dur, scene, min(dur, max_s)))
    return [(scene, length) for _, scene, length in sorted(scored, reverse=True)[:count]]

def render_vertical(master, start_s, length_s, subs, out):
    vf = (f"crop=ih*9/16:ih,scale=1080:1920,subtitles={subs.as_posix()}:"
          "force_style='FontName=Arial,FontSize=30,Outline=2,MarginV=90'")
    subprocess.run(["ffmpeg", "-y", "-ss", str(start_s), "-t", str(length_s), "-i", str(master), "-vf", vf,
                    "-c:v", "libx264", "-crf", "20", "-c:a", "aac", "-movflags", "+faststart", str(out)], check=True)
```

The center crop is the first version. A face-centered crop using the same face detector as the QC step is a planned improvement.

10.9 YouTube publisher

```
# studio/providers/publish/youtube.py
#   pip install google-api-python-client google-auth-oauthlib
from pathlib import Path
from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload
from studio.providers.base import Job, Result
```

```
SCOPES = ["https://www.googleapis.com/auth/youtube.upload", "https://www.googleapis.com/auth/youtube"]

class YouTubePublisher:
    def __init__(self, name, cfg):
        self.name, self.cfg = name, cfg
        self.secrets = Path(cfg.params.get("client_secrets", "secrets/youtube_client.json"))
        self.token = Path(cfg.params.get("token", "secrets/youtube_token.json"))

    def _creds(self):
        creds = Credentials.from_authorized_user_file(str(self.token), SCOPES) if self.token.exists() else None
        if not creds or not creds.valid:
            if creds and creds.expired and creds.refresh_token:
                creds.refresh(Request())
            else:
                creds = InstalledAppFlow.from_client_secrets_file(str(self.secrets), SCOPES).run_local_server(port=0)
            self.token.parent.mkdir(parents=True, exist_ok=True)
            self.token.write_text(creds.to_json())
        return creds

    def healthcheck(self) -> bool:
        return self.secrets.exists()

    def run(self, job: Job) -> Result:
        yt = build("youtube", "v3", credentials=self._creds())
        p = job.params
        body = {
            "snippet": {"title": p["title"], "description": p["description"], "tags": p.get("tags", []),
                        "categoryId": self.cfg.params.get("category_id", "1")},
            "status": {"privacyStatus": p.get("privacy", "private"), "selfDeclaredMadeForKids": False,
                       "containsSyntheticMedia": bool(p.get("synthetic_media", True))},   # the AI disclosure
        }
        media = MediaFileUpload(str(job.inputs["video"]), chunksize=8 * 1024 * 1024, resumable=True)
        request = yt.videos().insert(part="snippet,status", body=body, media_body=media)
        response = None
        while response is None:
            _, response = request.next_chunk()
        if p.get("playlist_id"):
            yt.playlistItems().insert(part="snippet", body={"snippet": {
                "playlistId": p["playlist_id"],
                "resourceId": {"kind": "youtube#video", "videoId": response["id"]}}}).execute()
        return Result(data={"video_id": response["id"], "url": f"https://youtu.be/{response['id']}"}, units=1)
```

The publish stage builds the description from the breakdown: a logline, chapter timestamps from the scene offsets, the AI-assistance note, and the playlist link. Uploads go up as private; you watch the episode once on YouTube, then switch it to scheduled or public from the review page.

10.10 The state store

```
# studio/store/base.py
from typing import Protocol

class Store(Protocol):
    def upsert_shot(self, episode: str, shot, scene_id: str) -> None: ...
    def get_shot(self, episode: str, shot_id: str) -> dict: ...
    def list_shots(self, episode: str) -> list[dict]: ...
    def set_status(self, episode: str, shot_id: str, stage: str, status: str, note: str = "") -> None: ...
    def record_usage(self, provider: str, units: float, cost_usd: float, ok: bool, kind: str | None = None) -> None:
...
    def usage_today(self, provider: str) -> tuple[int, float]: ...
    def quota_ok(self, provider: str) -> bool: ...
```

```
# studio/store/jsonfile.py
import json, os, tempfile
from datetime import date
from pathlib import Path

class JsonStore:
```

```
    def __init__(self, path: Path, limits: dict[str, dict]):
        self.path, self.limits = Path(path), limits
        self.data = json.loads(self.path.read_text()) if self.path.exists() else {"shots": {}, "usage": []}

    def _save(self):
        self.path.parent.mkdir(parents=True, exist_ok=True)
        fd, tmp = tempfile.mkstemp(dir=self.path.parent)        # atomic: never a half-written state file
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            json.dump(self.data, f, indent=2)
        os.replace(tmp, self.path)

    def upsert_shot(self, episode, shot, scene_id):
        key = f"{episode}/{shot.id}"
        row = self.data["shots"].setdefault(key, {"stages": {}})
        row.update(shot.model_dump(), episode=episode, scene=scene_id)
        self._save()

    def get_shot(self, episode, shot_id):
        return self.data["shots"][f"{episode}/{shot_id}"]

    def list_shots(self, episode):
        return [s for k, s in self.data["shots"].items() if k.startswith(episode + "/")]

    def set_status(self, episode, shot_id, stage, status, note=""):
        self.get_shot(episode, shot_id)["stages"][stage] = {"status": status, "note": note}
        self._save()

    def record_usage(self, provider, units, cost_usd, ok, kind=None):
        self.data["usage"].append({"day": date.today().isoformat(), "provider": provider, "units": units,
                                   "cost_usd": cost_usd, "ok": ok, "kind": kind})
        self._save()

    def usage_today(self, provider):
        rows = [u for u in self.data["usage"] if u["provider"] == provider and u["day"] == date.today().isoformat()
and u["ok"]]
        return len(rows), sum(u["cost_usd"] for u in rows)

    def quota_ok(self, provider):
        per_day = self.limits.get(provider, {}).get("per_day")
        return per_day is None or self.usage_today(provider)[0] < per_day
```

The SQLite store implements the same methods over three tables:

```
CREATE TABLE shots (episode TEXT, id TEXT, scene TEXT, data JSON, PRIMARY KEY (episode, id));
CREATE TABLE stage_status (episode TEXT, shot_id TEXT, stage TEXT, status TEXT, note TEXT,
                           updated_at TEXT, PRIMARY KEY (episode, shot_id, stage));
CREATE TABLE usage (day TEXT, provider TEXT, units REAL, cost_usd REAL, ok INTEGER, kind TEXT, at TEXT);
CREATE TABLE approvals (episode TEXT, shot_id TEXT, stage TEXT, decision TEXT, note TEXT, at TEXT);
```

The factory reads `storage.backend` and the provider limits:

```
# studio/store/__init__.py
def make_store(config):
    limits = {name: p.limits for name, p in config.providers.items()}
    path = config.root / config.pipeline.storage.path
    if config.pipeline.storage.backend == "json":
        from studio.store.jsonfile import JsonStore; return JsonStore(path, limits)
    from studio.store.sqlite import SqliteStore; return SqliteStore(path, limits)
```

10.11 The command line

```
# studio/cli.py      pip install typer python-dotenv
from pathlib import Path
import typer
from dotenv import load_dotenv
from studio.config import load_config
from studio.store import make_store
```

```
app = typer.Typer(help="AI series studio pipeline", no_args_is_help=True)

@app.callback()
def _init(ctx: typer.Context, root: Path = typer.Option(Path("."), help="project folder")):
    load_dotenv(root / ".env")
    cfg = load_config(root)
    ctx.obj = {"config": cfg, "store": make_store(cfg)}

@app.command("config-check")
def config_check(ctx: typer.Context):
    """Validate config, ping every enabled provider, print the effective chain per stage."""
    from studio.providers.registry import build
    cfg = ctx.obj["config"]
    for name, p in cfg.providers.items():
        health = build(name, p).healthcheck() if p.enabled else "-"
        typer.echo(f"{name:20} {p.type:18} enabled={str(p.enabled):5} healthy={health}")
    for stage, s in cfg.pipeline.stages.items():
        active = [n for n in s.chain if cfg.providers[n].enabled]
        typer.echo(f"stage {stage:10} chain={active}")

@app.command()
def breakdown(ctx: typer.Context, episode: str, provider: str = typer.Option(None, "--provider")):
    from studio.stages import breakdown as st
    _guard(lambda: st.run(ctx.obj["config"], ctx.obj["store"], episode, provider))

@app.command()
def run(ctx: typer.Context, stage: str, episode: str, shots: str = "", provider: str = typer.Option(None, "--
provider")):
    """Run one stage: keyframes | video | voice | lipsync | music. --shots s03_07,s03_08 limits it."""
    from studio.stages.runner import run_stage
    report = run_stage(ctx.obj["config"], ctx.obj["store"], stage, episode, shots.split(",") if shots else None,
provider)
    _print_report(report)

@app.command()
def resume(ctx: typer.Context, episode: str):
    """Collect manual and remote results, then continue every stage that has work."""
    from studio.stages.runner import resume as _resume
    _print_report(_resume(ctx.obj["config"], ctx.obj["store"], episode))

@app.command()
def review(ctx: typer.Context, port: int = 8787):
    import uvicorn
    from studio.review.app import create_app
    uvicorn.run(create_app(ctx.obj["config"], ctx.obj["store"]), host="127.0.0.1", port=port)

def _print_report(report: dict):
    for k, v in report.items():
        if v:
            typer.echo(f"{k:8} {len(v):4}  {', '.join(v[:12])}{' ...' if len(v) > 12 else ''}")
    if report.get("manual"):
        typer.secho("Manual steps are waiting. Open the review page (studio review) or the MANUAL_REQUIRED.md
files.", fg="yellow")

def _guard(fn):
    from studio.chain import StageBlocked
    from studio.providers.base import ManualRequired
    try:
        fn()
    except ManualRequired as e:
        typer.secho(f"Manual step: open {e.request_file} and save the result as {e.expected_output}", fg="yellow")
    except StageBlocked as e:
        typer.secho(str(e), fg="red"); raise typer.Exit(2)

if __name__ == "__main__":
    app()
```

Other commands in the full CLI, not shown: `assemble`, `cutdowns`, `publish --dry-run`, `sync` (rclone both ways), `worker` (local alias for the worker script), `usage` (today's ledger per provider), `store migrate`, `bible validate`, `evidence export`.

10.12 The review page

FastAPI with HTMX and no build step. Pages:

Route	Shows	Actions
`/`	Episodes and a status count per stage	open an episode
`/ep/{ep}`	Grid of shots with a status chip per stage, filter by status	bulk approve, run stage
`/ep/{ep}/shot/{id}`	Keyframe candidates side by side with the reference sheet, clip attempts with QC notes, voice lines with play buttons	approve, reject with note, regenerate (note goes into the prompt), move to premium lane, force provider
`/ep/{ep}/manual`	Every pending manual request with the prompt and a copy button	drag-and-drop upload saved under the expected name, then resume
`/usage`	Calls, units and cost per provider per day; the daily quota bar for the free tiers	export CSV
`/ep/{ep}/publish`	The assembled master, description preview with chapters, disclosure checkbox	upload private, set public

```
# studio/review/app.py  (the approve route, as a sample of the pattern)
from datetime import datetime
from fastapi import FastAPI, Form
from fastapi.responses import HTMLResponse

def create_app(config, store) -> FastAPI:
    app = FastAPI()

    @app.post("/ep/{ep}/shot/{sid}/{stage}/approve", response_class=HTMLResponse)
    def approve(ep: str, sid: str, stage: str, candidate: str = Form(...), note: str = Form("")):
        shot_dir = config.root / "episodes" / ep / "shots" / sid / stage
        src = shot_dir / candidate
        dest = shot_dir / ("approved" + src.suffix)
        dest.write_bytes(src.read_bytes())
        store.set_status(ep, sid, stage, "approved", candidate)
        store.record_approval(ep, sid, stage, "approve", note, datetime.now().isoformat())
        return f'<span class="chip ok">approved {candidate}</span>'

    return app
```

Every approve and reject is written to the `approvals` table. `studio evidence export` turns that table, the script versions and the breakdown edits into a zip per episode: the human-direction record for any platform review.

11. Stage specifications

Stage	Input	Output	Default chain	QC	Gate	Manual fallback
script	bible, season outline	`episodes/<ep>/script.md`	manual_text	none	you wrote it	Paste the script from any chat app
breakdown	script, bible, style	`breakdown.json`, one shot record per shot	gemini_free, claude, openai, manual_text	schema validation	you edit the JSON	Paste the prompt into a chat app, save `response.json`
keyframes	shot, character and location sheets, style	4 candidates per shot, ranked	comfy_qwen_image, manual_image	face or CLIP similarity	approve one	Generate in Nano Banana Pro, save as `cand_1.png`
video	approved keyframe, video prompt	up to 4 attempts, first keeper offered	comfy_wan22, comfy_ltx23, manual_video	duration, black, freeze	approve	Generate in Flow, save as `attempt_1.mp4`
voice	lines, character voice reference	`line_n.wav` per line, normalized	chatterbox_local, kokoro_local, manual_audio	duration sanity, loudness	spot check per character	Record or generate elsewhere, save as `line_n.wav`
lipsync	approved clip, line audio	`approved.mp4` with lip sync	comfy_latentsync, manual_video	duration match	spot check	Save a synced clip as `lipsync/attempt_1.mp4`
music	scene moods	one cue per scene	comfy_acestep, manual_music	duration	spot check	Lyria 3 or YouTube Audio Library, save as `cue.wav`
assemble	all approved assets	`out/<ep>.mp4`, `.srt`	FFmpeg	loudness, length	watch the episode	n/a
cutdowns	master, breakdown	3 vertical clips	FFmpeg	length	pick which to post	n/a
publish	master, metadata	YouTube video id	youtube	dry run first	confirm, then public	Upload by hand in YouTube Studio with the disclosure ticked

11.1 Prompt templates for images and video

```
# prompts/keyframe.md
{{ style.one_line }}
{{ shot.shot_type }} shot. {{ shot.action }}
{% for c in characters %}{{ c.name }}: {{ c.look }}. {% endfor %}
Location: {{ location.look }}. {{ scene.time_of_day }}, mood {{ scene.mood }}.
Camera: {{ shot.camera }}. 16:9, clean composition, no text, no watermark.
{% if reviewer_note %}Correction from the previous attempt: {{ reviewer_note }}{% endif %}
```

```
# prompts/video.md
{{ shot.action }} Camera: {{ shot.camera }}. {{ style.motion_words }}
Keep the characters exactly as in the input image. Smooth motion, no morphing, no extra people, no text.
```

Character and location "look" strings live in the bible as short, fixed descriptions that every prompt repeats word for word. Repeating the same words is a large part of consistency with open models, and the LoRA does the rest.

11.2 Evidence of human direction

The pipeline produces this record for every episode without extra work:

- the human-written script with its edit history in git,
- the breakdown prompt and the edited breakdown,
- every prompt, seed and provider used per shot,
- the approvals table with timestamps and reviewer notes,
- the reviewer corrections that were fed back into prompts.

`studio evidence export <ep>` zips it. Keep it with the episode.

12. Build plan

Estimates assume one developer who knows Python, working part time. Each milestone ends with something you can run.

Milestone	Scope	Hours	Acceptance test
M0 Skeleton	repo, `pyproject.toml`, config loader, schemas, `.env`, `config-check`, JSON store	5 to 7	`studio config-check` lists providers and chains; a bad chain fails validation
M1 Breakdown and manual lane	prompt template, Gemini, Claude and OpenAI adapters, manual provider, chain runner, usage ledger, `breakdown` and `resume` commands	8 to 10	Pilot script becomes a valid `breakdown.json` through Gemini; with the key removed, the manual path produces the same result
M2 Keyframes and review	ComfyUI client, Qwen-Image-Edit workflow, candidates and ranking, review page with approve, reject, regenerate, manual upload	12 to 16	Every pilot shot has an approved keyframe; a Nano Banana Pro image imported by hand flows through unchanged
M3 Video, worker, QC	Wan 2.2 workflow, QC, retries with new seeds, remote queue, Kaggle notebook, `sync`	16 to 20	Pilot clips generated on Kaggle through the queue; a failed clip is retried and a QC rejection is visible in the UI
M4 Voice and assembly	Chatterbox and Kokoro adapters, dialogue track, SRT, `render_shot`, concat, mix and master	10 to 14	`pilot.mp4` plays end to end with dialogue, music ducking and subtitles
M5 Lip sync, music, cutdowns	LatentSync workflow, ACE-Step workflow, scene cues, vertical cutdowns	10 to 12	Dialogue close-ups are synced; three vertical clips exist with captions
M6 Publish and dashboard	YouTube OAuth, upload with chapters and disclosure, usage page, evidence export	6 to 8	Pilot uploaded as private with the synthetic media flag set; evidence zip produced
M7 Hardening	SQLite store, LoRA training script per character, overnight batch mode, face-centered cutdowns, tests	ongoing	A full 30-minute episode runs overnight with no manual intervention except the gates

Total for M0 to M6: 67 to 87 hours, which is five to seven weeks part time. The pilot is finished at M6. The GPU decision happens there.

12.1 Order of work inside each milestone

1. Write the acceptance test first as a script in `tests/`, even if it only checks that files exist.
2. Build the manual path before the API path. If the manual path works, the stage is usable on day one.
3. Add the API or model provider, then confirm the chain falls back to manual when the provider is disabled.
4. Add the review page for the stage last; the CLI output is enough to test with.

12.2 Paid upgrades, in order, once revenue arrives

1. Kling Standard at $10 a month for hero shots that Wan cannot nail. Add the adapter, set `enabled: true`, put it first in the video chain for shots on the premium lane.
2. ElevenLabs Starter at $6 a month if a lead character needs a voice with more range than Chatterbox gives.
3. Google AI Pro at $20 a month when the student plan ends, for Nano Banana Pro consistency on keyframes.
4. A second GPU or a cloud GPU by the hour when a weekly episode no longer fits in overnight batches.

13. Runbook

13.1 One-time setup on the laptop (Windows)

```
winget install Python.Python.3.12 Git.Git Gyan.FFmpeg
pip install uv
git clone https://github.com/<you>/ai-series-studio && cd ai-series-studio
uv venv && .venv\Scripts\activate && uv pip install -e .
copy .env.example .env          (then paste GEMINI_API_KEY)
studio config-check
```

For the local ComfyUI (optional until the GPU arrives): clone ComfyUI, install its requirements, run `scripts/get_models.ps1` to download Qwen-Image-Edit, Wan 2.2 and LatentSync weights, start it with `python main.py --listen 127.0.0.1 --port 8188`.

13.2 Accounts

1. Google AI student plan: verify student status on Google's student offer page, redeem before 31 December 2026.
2. Gemini API key: Google AI Studio, "Get API key", no billing account.
3. YouTube: create the channel, then in Google Cloud Console create a project, enable the YouTube Data API v3, create an OAuth client of type Desktop, download it as `secrets/youtube_client.json`. The first `studio publish` opens a browser for consent. The daily quota of 10,000 units allows about six uploads a day.
4. Kaggle: enable phone verification to unlock GPU sessions; add the rclone config as a secret named `RCLONE_CONF`.
5. rclone: `rclone config` once on the laptop with a Google Drive remote named `gdrive`; copy the resulting config text into the Kaggle secret.
6. GitHub Student Developer Pack for Copilot Pro.

13.3 Producing an episode

```
1  write episodes/ep01/script.md                    (chat app of your choice, then edit by hand)
2  studio breakdown ep01                            (Gemini; or paste if the quota is gone)
3  edit episodes/ep01/breakdown.json                (merge shots, fix names, mark premium lane)
4  studio run keyframes ep01  ->  studio review     (approve one candidate per shot)
5  studio run video ep01      ->  studio sync       (jobs go to Drive)
6  run the Kaggle notebook                          (worker drains the queue, 9 h max per session)
7  studio sync  ->  studio resume ep01  ->  review  (approve clips; regenerate with a note if needed)
8  studio run voice ep01 ; studio run lipsync ep01 ; studio run music ep01
9  studio assemble ep01 ; watch out/ep01.mp4
10 studio cutdowns ep01
11 studio publish ep01 --dry-run ; studio publish ep01   (private) ; set public from the review page
12 studio evidence export ep01
```

Manual steps appear in the review page under "Manual" and as `MANUAL_REQUIRED.md` files. Finish them, then `studio resume`.

13.4 Daily rhythm during production

- Morning: review overnight results, approve, mark regenerations with notes.
- Midday: run voice and lip sync on the laptop CPU while the GPU handles video.
- Evening: queue the next batch; on Kaggle start the session before the 9-hour window you need.
- Weekly: one episode published, three cutdowns posted, usage ledger checked against the free-tier limits.

13.5 Troubleshooting

Symptom	Likely cause	Fix
`config-check` shows a provider unhealthy	missing key in `.env`, ComfyUI not running, wrong `base_url`	`studio config-check` prints the reason; fix and re-run
Breakdown fails validation	the model dropped fields or invented a character	the chain retries once; edit the response file by hand if it reaches the manual lane
Character drifts across shots	look strings differ between prompts, or no LoRA yet	lock the look string in the bible; train the LoRA (M7)
Kaggle session ended mid-batch	9-hour limit or idle timeout	the worker is restartable; unfinished jobs are still in `pending` after `sync`
Clip rejected for freeze	model produced a static result	automatic retry with a new seed; if 4 fail, mark the shot as an animated still
Upload returns 403	quota exhausted or consent expired	wait a day or delete `secrets/youtube_token.json` and consent again

14. Compliance, licensing and disclosure

14.1 Disclosure per platform

Platform	What to do	Where the pipeline does it
YouTube	Mark "altered or synthetic content" on upload; keep the human script and approvals	`containsSyntheticMedia: true` in the publish stage; evidence export
Instagram, Facebook	Apply the AI label on each post	Manual, in the composer; the cutdown description includes the note
TikTok	Apply the AI-generated content label	Manual, in the composer

14.2 License table

Component	License	Commercial use of output
Wan 2.2	Apache 2.0	yes
LTX-2.3	LTX-2 community license	yes, within its revenue cap; check the current terms before enabling
HunyuanVideo 1.5	Tencent community license	yes outside the excluded regions; not needed for this plan
Qwen-Image, Qwen-Image-Edit	Apache 2.0	yes
Flux.1 schnell	Apache 2.0	yes
Flux.1 dev, Flux Kontext dev	non-commercial	no
Chatterbox	MIT	yes; outputs carry an inaudible watermark, which is fine
Kokoro	Apache 2.0	yes
LatentSync, InfiniteTalk	Apache 2.0	yes
ACE-Step 1.5	Apache 2.0, trained on licensed and royalty-free data	yes
YouTube Audio Library	YouTube's license	yes, monetization safe
Freesound CC0, Pixabay	CC0 or Pixabay license	yes
Gemini, Nano Banana Pro, Veo, Lyria on the student plan	Google's generative AI terms	yes for your own generated content; do not automate the app
ComfyUI	GPL-3.0 for the server code	outputs are yours; the server stays unmodified

14.3 What keeps the channel on the right side of the July 2026 YouTube policy

- An original script written by a person for every episode, versioned in git.
- Varied shots, real story structure, recurring characters with arcs. Nothing templated.
- No real-person voice cloning. Chatterbox references are either your own recorded voice or synthetic seeds.
- No AI persona discussing health, finance or law.
- Disclosure on every upload, and the evidence zip ready if a review ever asks.

15. Risks and mitigations

Risk	Impact	Mitigation
Character drift across shots	Viewers notice; the series looks cheap	Fixed look strings, reference sheets in every prompt, LoRA per character, similarity QC before review
Kaggle throttling or policy change	Pilot slows down	Colab as overflow; the worker is portable; the GPU purchase is the real fix
Free-tier quota exhausted mid-run	Stage stalls	Daily limits in config, ledger checks before every call, manual lane always last
YouTube reviews the channel under the inauthentic content policy	Demonetization	Human script and gates, disclosure flag, evidence export; no mass-production patterns
Facebook and TikTok never pay in Pakistan	Lower upside	Plan already assumes YouTube only; the others are reach channels
Load shedding during overnight batches	Lost work	Worker saves per clip and resumes; a UPS for the router and the laptop; the GPU box can wait
Model updates break a workflow	Generation errors	Workflows are pinned JSON files in git; update one at a time with the pilot as the test
Scope creep in the software	Pilot delayed	Milestones with acceptance tests; the manual path first; nothing beyond M6 before the pilot ships
Student plan ends after a year	Lose Nano Banana Pro and Veo	Those are the premium lane only; the free lane carries the series

16. Appendix

16.1 Command reference

Command	Purpose
`studio config-check`	Validate config, ping providers, print chains
`studio breakdown <ep> [--provider name]`	Script to breakdown
`studio run <stage> <ep> [--shots a,b] [--provider name]`	Run keyframes, video, voice, lipsync or music
`studio resume <ep>`	Collect manual and remote results and continue
`studio review [--port 8787]`	Open the local review page
`studio sync`	rclone the queue to and from Drive
`studio worker [--once]`	Run the worker against the local ComfyUI
`studio assemble <ep>`	Render the master and SRT
`studio cutdowns <ep>`	Render the vertical clips
`studio publish <ep> [--dry-run]`	Upload to YouTube as private
`studio usage [--day YYYY-MM-DD]`	Provider usage and cost ledger
`studio store migrate --to sqlite`	Move state between backends
`studio evidence export <ep>`	Zip the human-direction record

16.2 Dependencies

```
# pyproject.toml [project.dependencies]
pydantic>=2.7, pyyaml, jinja2, typer, python-dotenv, websocket-client, pydub,
fastapi, uvicorn, python-multipart, jinja2,
google-genai, anthropic, openai,
google-api-python-client, google-auth-oauthlib,
# optional extras
[qc]      insightface, onnxruntime, opencv-python, open_clip_torch
[tts]     chatterbox-tts, torchaudio, kokoro, soundfile
[sqlite]  (standard library)
# worker machine
torch (CUDA build), ComfyUI and its requirements, rclone
```

16.3 Sources consulted for the facts in this plan

- YouTube monetization policy timeline 2026 (AIR Media-Tech) and the July 2026 inauthentic content update (The Desk, Hackernoon)
- TikTok Creator Rewards and AI restrictions (Storrito, Kineclip)
- Facebook Content Monetization 2026 guide (ShortSync) and AI content rules (Cinerads)
- Instagram Reels monetization status 2026 (Lilach Bullock, Virvid)
- Amazon Prime Video Direct submissions (CineD, No Film School)
- AI video API pricing 2026 (FluxNote, BuildMVPFast, Curionic comparison)
- Open-source video models and VRAM (Thunder Compute, AI Magicx, Best Free Web Resources ComfyUI guide)
- Kling free tier terms (AIReiter), Runway free plan (Runway help center), Grok Imagine changes (BudgetPixel)
- Google AI student offers for Pakistan (ProPakistani, APP), Google AI Plus features (9to5Google), Gemini limits (Zenken, Yingtu, LaoZhang)
- Free GPU options (GPU per Hour), open-source TTS comparison (AI Video Sensei, The Decoder), ACE-Step 1.5 (ComfyUI Wiki)

- GPU prices in Pakistan (OLX, TechMatched, Qeemat), YouTube and Facebook availability in Pakistan (PKRevenue, ProPakistani, Express Tribune)